

# PostgreSQL Production Labs

Complete Session Companion — all 42 sessions

Every session explored to production depth: the mechanism, the configuration, the commands, and the failure mode it exists to prevent. Written to be used at the terminal, not read once.

42

SESSIONS

5

PHASES

20

CORE:  
ARCHITECTURE

9

BACKUP & HA

|                 |                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------|
| SCOPE           | PostgreSQL 14 – 17, self-managed and AWS-managed                                                                              |
| SAMPLE DATABASE | xxxx, with objects prefixed xxxx_                                                                                             |
| PLATFORMS       | RHEL / Rocky, Ubuntu / Debian, AWS RDS and Aurora                                                                             |
| TOOLING COVERED | pgBackRest, Patroni, PgBouncer, PgPool-II, pgBadger, ora2pg, DMS, Terraform, Ansible, Liquibase, Jenkins, Prometheus, Grafana |
| PREPARED        | September 2026                                                                                                                |
| CLASSIFICATION  | Internal working document                                                                                                     |

## PHASE 1

# PostgreSQL Core: Architecture, Installation and Administration

Sessions 1–20

---

Why PostgreSQL works the way it does. Every operational decision in the later phases follows from the mechanisms established here.

**01** Why PostgreSQL — History, MVCC, WAL, Architecture Overview

---

**02** Connection Establishment — Part 1 — The postmaster process (theory)

---

**03** Connection Establishment — Part 2 — Hands-on scenarios

---

**04** ACID Properties — Transactions, isolation, atomicity in depth

---

**05** Installation — Part 1 — Source Code, RPM, YUM

---

**06** Installation — Part 2 — Extensions and the devel module

---

**07** Installation — Part 3 — Ubuntu, EDB, Percona Distribution

---

**08** Installation — Part 4 — Scenarios and troubleshooting

---

**09** Architecture — The CRUD Workflow end to end

---

**10** Logger Process — How PostgreSQL logging works

---

**11** Autovacuum Part 1 — What it does and why it matters

---

**12** Autovacuum Part 2 — Tuning the parameters

---

**13** Autovacuum Part 3 — Bloat detection and remediation

---

**14** Post-Installation — pg\_hba.conf and SCRAM-SHA-256

---

**15** psql — Command-line mastery for DBAs

---

**16** WAL Files — Write-Ahead Logging internals and management

---

**17** Datafiles and Control Files — Physical structure

---

**18** Shutdown Modes — Smart, fast, immediate

---

**19** User Administration 1 — Roles, users, privileges

---

**20** User Administration 2 — RBAC and Row-Level Security

---

# 01 Why PostgreSQL — History, MVCC, WAL, Architecture Overview

## CORE

**Why this comes first.** Almost every operational decision you will make later — why VACUUM exists, why a `DELETE` does not free space, why replication ships log rather than rows — follows from three design choices made decades ago. Learn them once and the rest of the course stops being memorisation.

## Lineage

POSTGRES began at UC Berkeley in 1986 under Michael Stonebraker as the successor to Ingres. SQL support arrived in 1994 (Postgres95), and the project was renamed PostgreSQL in 1996 when it moved to community governance. That governance model — no single corporate owner, a global development group, an annual major release each autumn — is the reason the licence is permissive and the reason the roadmap is not anyone's product strategy.

## MVCC — Multi-Version Concurrency Control

PostgreSQL never overwrites a row in place. An `UPDATE` writes a **new tuple version** and marks the old one as expired. A `DELETE` only marks. Every tuple carries hidden system columns:

| COLUMN            | MEANING                                                  |
|-------------------|----------------------------------------------------------|
| <code>xmin</code> | Transaction ID that inserted this version                |
| <code>xmax</code> | Transaction ID that deleted or superseded it (0 if live) |
| <code>ctid</code> | Physical location: (block number, offset within block)   |

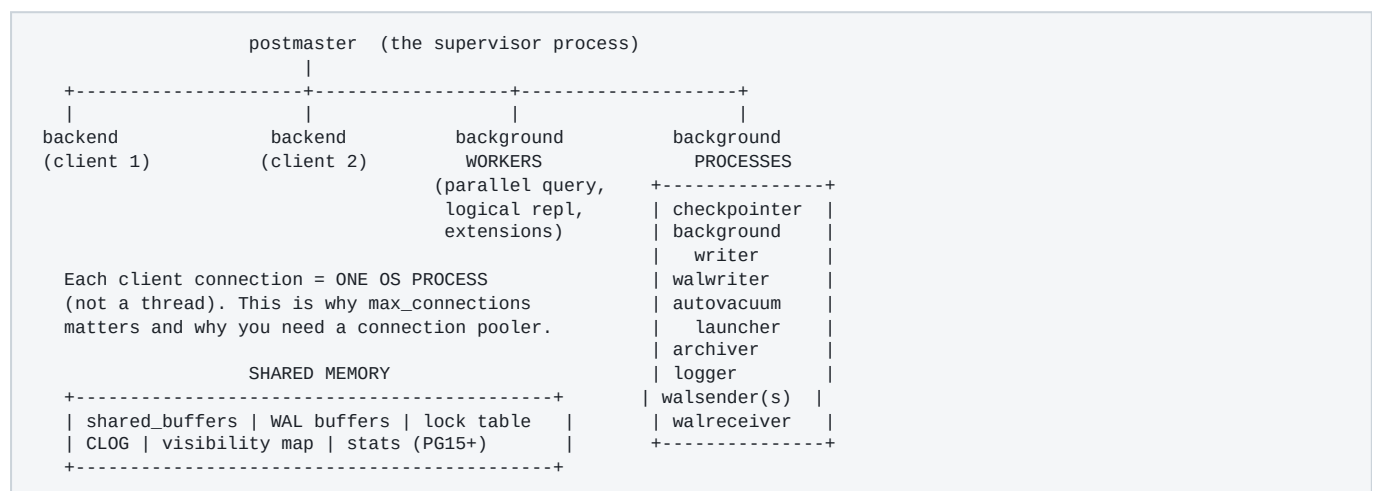
```
SELECT xmin, xmax, ctid, * FROM xxxx_orders LIMIT 5;
```

A transaction sees a tuple if `xmin` is committed and visible to its snapshot, and `xmax` is either zero or not visible. The consequence that matters operationally: **readers never block writers and writers never block readers**, at the cost of dead tuples accumulating on disk. That cost is what VACUUM exists to pay.

## WAL — Write-Ahead Logging

Before any change reaches a data file, its description is written to the write-ahead log and flushed. This single rule delivers crash recovery, point-in-time recovery and replication, all from the same mechanism. If the server dies, replaying WAL from the last checkpoint reconstructs a consistent state. If you ship WAL somewhere else, you have a replica. If you keep WAL, you have PITR.

## Process architecture



#### THE ONE-PROCESS-PER-CONNECTION CONSEQUENCE

Every connection forks a full OS process with its own memory. A thousand idle connections is a thousand processes, each holding relcache and catcache. This is the single biggest architectural difference from MySQL or Oracle's shared-server mode, and it is why **PgBouncer is not optional** at scale — a point Session 28 returns to.

## 02 Connection Establishment — Part 1 — The postmaster process (theory)

CORE

What happens between a client calling `connect()` and being able to run a query. Understanding this sequence turns most connection failures from mysteries into a single obvious step.



### What the postmaster does and does not do

| DOES                                     | DOES NOT                      |
|------------------------------------------|-------------------------------|
| Listens on the socket and TCP port       | Execute any SQL               |
| Performs authentication                  | Touch your data               |
| Forks backend processes                  | Hold user session state       |
| Starts and restarts background processes | Participate in the query path |
| Handles crash recovery on startup        | —                             |

If a backend crashes, the postmaster detects it, terminates all other backends and initiates crash recovery — because a crashed backend may have left shared memory inconsistent. This is why one bad query can restart the whole instance, and why you will see `terminating connection because of crash of another server process` in the log.

## The settings that govern all of this

```
listen_addresses = '*'           -- which interfaces; 'localhost' is the default and
                                -- the most common cause of "connection refused"
port = 5432
max_connections = 200           -- static: requires a restart
superuser_reserved_connections = 3
unix_socket_directories = '/var/run/postgresql'
ssl = on
password_encryption = 'scram-sha-256'
authentication_timeout = '1min'
tcp_keepalives_idle = 60
```

### PG\_HBA.CONF: FIRST MATCH WINS

Evaluation stops at the first line whose TYPE, DATABASE, USER and ADDRESS all match. A broad `host all all 0.0.0.0/0 md5` placed above your specific rule means the specific rule is never reached. Order from most specific to least specific, always — and verify with `SELECT * FROM pg_hba_file_rules ORDER BY line_number;`, which also exposes malformed lines in its `error` column.

## 03 Connection Establishment — Part 2 — Hands-on scenarios

### CORE

The same sequence, seen from the diagnostic side. Each failure mode below maps to exactly one step of the handshake in Session 2.

| SYMPTOM                                        | FAILED AT      | FIRST CHECK                                                               |
|------------------------------------------------|----------------|---------------------------------------------------------------------------|
| Connection refused                             | TCP connect    | Service down, wrong port, or <code>listen_addresses</code>                |
| Hangs then times out                           | TCP connect    | Firewall, security group, routing                                         |
| no <code>pg_hba.conf</code> entry for host     | Rule lookup    | <code>pg_hba_file_rules</code> — no matching line                         |
| password authentication failed                 | Authentication | Wrong password, or an md5 hash with a scram rule                          |
| <code>fe_sendauth: no password supplied</code> | Authentication | Client sent nothing — use <code>PGPASSWORD</code> or <code>.pgpass</code> |
| role "x" does not exist                        | Startup packet | Role missing or misspelled                                                |
| database "x" does not exist                    | Startup packet | Wrong database name                                                       |
| permission denied for database                 | Post-auth      | No <code>CONNECT</code> privilege                                         |
| too many clients already                       | Fork           | <code>max_connections</code> exhausted                                    |
| too many connections for role                  | Fork           | Role <code>CONNECTION LIMIT</code>                                        |

### Working the problem from the outside in

```
# 1. Is anything listening?
ss -lntp | grep 5432
sudo -u postgres psql -c "SHOW listen_addresses; SHOW port;"

# 2. Reachable from the client host?
nc -zv db.host 5432

# 3. Which rule would match?
psql -c "SELECT line_number, type, database, user_name, address, auth_method, error
        FROM pg_hba_file_rules ORDER BY line_number;"

# 4. Does the role exist and can it log in?
psql -c "SELECT rolname, rolcanlogin, rolvaliduntil, rolconnlimit
        FROM pg_roles WHERE rolname = 'xxxx_app';"

# 5. Connection headroom
psql -c "SELECT count(*) AS used, current_setting('max_connections') AS max,
        count(*) FILTER (WHERE state='idle') AS idle,
        count(*) FILTER (WHERE state='idle in transaction') AS idle_in_txn
        FROM pg_stat_activity;"

# 6. The server log is authoritative — it records the reason the client is not told
tail -100 $PGDATA/log/postgresql-*.log
```

## Connection-level settings worth knowing

```
-- Per-role defaults, applied at connect time
ALTER ROLE xxxx_report SET statement_timeout = '5min';
ALTER ROLE xxxx_report SET default_transaction_read_only = on;
ALTER ROLE xxxx_app CONNECTION LIMIT 50;
ALTER ROLE xxxx_app VALID UNTIL '2027-01-01';

-- Per-database defaults
ALTER DATABASE xxxx SET timezone = 'Asia/Kolkata';

-- What is actually in force for this session, and where it came from
SELECT name, setting, source, sourcefile, sourceline
FROM   pg_settings WHERE name = 'statement_timeout';
```

### A HABIT WORTH FORMING

Fix the layer the error names, not the layer you assume. *No `pg_hba.conf` entry* is never a password problem, and *connection refused* is never a permissions problem. The message is precise; read it before changing anything.



## 04 ACID Properties — Transactions, isolation, atomicity in depth

### CORE

How PostgreSQL actually delivers each letter — because the mechanisms explain the operational behaviour you will meet later.

| PROPERTY           | MECHANISM IN POSTGRESQL                                                                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Atomicity</b>   | Transaction status in <code>pg_xact</code> (CLOG). A transaction becomes visible the instant its commit record is flushed; there is no partial state to undo because nothing was overwritten. |
| <b>Consistency</b> | Constraints, triggers, foreign keys, and the transaction boundary itself.                                                                                                                     |
| <b>Isolation</b>   | MVCC snapshots plus, at the highest level, Serializable Snapshot Isolation (SSI).                                                                                                             |
| <b>Durability</b>  | WAL flushed by <code>fsync</code> before commit returns. <code>synchronous_commit</code> controls how far that flush must reach.                                                              |

The four isolation levels as PostgreSQL implements them

| LEVEL                           | DIRTY READ   | NON-REPEATABLE READ | PHANTOM      | SERIALIZATION ANOMALY |
|---------------------------------|--------------|---------------------|--------------|-----------------------|
| Read uncommitted                | Not possible | Possible            | Possible     | Possible              |
| <b>Read committed</b> (default) | Not possible | Possible            | Possible     | Possible              |
| Repeatable read                 | Not possible | Not possible        | Not possible | Possible              |
| Serializable                    | Not possible | Not possible        | Not possible | Not possible          |

### TWO POSTGRESQL-SPECIFIC FACTS WORTH STATING IN AN INTERVIEW

**Read uncommitted behaves exactly as read committed** — dirty reads are impossible in PostgreSQL because MVCC has no mechanism to expose an uncommitted tuple version. And **repeatable read already prevents phantoms**, which the SQL standard does not require; PostgreSQL's snapshot isolation is stronger than the standard's definition of that level.

### Snapshot behaviour, demonstrated

```
-- Read committed: each STATEMENT takes a fresh snapshot
BEGIN;                                     -- session A
SELECT count(*) FROM xxxx_orders;         -- 1000
--      (session B inserts a row and commits)
SELECT count(*) FROM xxxx_orders;         -- 1001 <- changed
COMMIT;

-- Repeatable read: the TRANSACTION takes one snapshot at first statement
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM xxxx_orders;         -- 1000
--      (session B inserts a row and commits)
SELECT count(*) FROM xxxx_orders;         -- 1000 <- stable
COMMIT;
```

### Serializable and write skew

Serializable uses SSI: it permits transactions to run concurrently and **aborts one at commit time** if the interleaving could not have arisen from any serial order. Applications using it must handle SQLSTATE `40001` by retrying.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
-- ... work ...
COMMIT; -- may raise: ERROR: could not serialize access due to read/write dependencies
```

Durability controls

| SYNCHRONOUS_COMMIT | COMMIT RETURNS ONCE WAL IS...       | DATA LOSS WINDOW           |
|--------------------|-------------------------------------|----------------------------|
| off                | in the WAL buffer, not yet flushed  | Up to wal_writer_delay × 3 |
| local              | flushed to local disk               | None locally               |
| remote_write       | written to the standby's OS         | OS crash on standby        |
| on (default)       | flushed to the standby's disk       | None                       |
| remote_apply       | replayed and visible on the standby | None; highest latency      |

```
-- Per-transaction, which is where this usually belongs
SET LOCAL synchronous_commit = off; -- e.g. a bulk load where a few seconds' loss is acceptable
```

**NEVER TURN OFF FSYNC**

`fsync = off` makes benchmarks look wonderful and makes a power loss unrecoverable — not “some data lost” but a corrupted cluster. `synchronous_commit = off` is the safe way to trade a small durability window for throughput: it risks the last few transactions, never the cluster's integrity.

## 05 Installation — Part 1 — Source Code, RPM, YUM

### CORE

From source — when you need a build option the packages do not carry

```
sudo dnf install -y gcc make readline-devel zlib-devel openssl-devel \
                    libicu-devel bison flex perl-ExtUtils-Embed \
                    libxml2-devel libxslt-devel systemd-devel llvm-devel clang

curl -O https://ftp.postgresql.org/pub/source/v17.0/postgresql-17.0.tar.bz2
tar xjf postgresql-17.0.tar.bz2 && cd postgresql-17.0

./configure --prefix=/usr/local/pgsql17 \
            --with-openssl --with-libxml --with-libxslt \
            --with-systemd --with-llvm --with-icu \
            --with-pgport=5432

make -j$(nproc) world
sudo make install-world          # includes contrib and documentation
```

`make world` / `install-world` rather than plain `make install` is the difference between having the contrib extensions available and discovering later that `pg_stat_statements` is not there.

### Cluster initialisation

```
sudo useradd -m -s /bin/bash postgres
sudo mkdir -p /data/pgsql17 && sudo chown postgres:postgres /data/pgsql17
sudo chmod 700 /data/pgsql17

sudo -u postgres /usr/local/pgsql17/bin/initdb \
  -D /data/pgsql17 \
  --encoding=UTF8 --locale=en_US.UTF-8 \
  --data-checksums \
  --auth-host=scram-sha-256 --auth-local=peer \
  --pwprompt
```

#### --DATA-CHECKSUMS CANNOT BE ADDED LATER WITHOUT A REBUILD

It detects silent storage corruption and is a prerequisite for `pg_rewind` unless you also set `wal_log_hints`. The overhead is a few percent. Turn it on at `initdb` every time — retrofitting means `pg_checksums` with the cluster offline, or a dump and reload.

### RPM and the PGDG YUM repository — the normal production route

```
# 1. Add the PGDG repository
sudo dnf install -y https://download.postgresql.org/pub/repos/yum/reporgms/EL-9-x86_64/pgdg-redhat-repo-latest.noarch.rpm

# 2. Disable the distribution's own module, which would shadow PGDG
sudo dnf -qy module disable postgresql

# 3. Install
sudo dnf install -y postgresql17-server postgresql17-contrib postgresql17-devel

# 4. Initialise and start
sudo /usr/pgsql-17/bin/postgresql-17-setup initdb
sudo systemctl enable --now postgresql-17
sudo systemctl status postgresql-17
```

| PACKAGE              | CONTAINS                                                                                                      |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| postgresql17-server  | The server, <code>initdb</code> , <code>pg_ctl</code> , the <code>systemd</code> unit                         |
| postgresql17         | Client binaries: <code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code>                           |
| postgresql17-contrib | <code>pg_stat_statements</code> , <code>pgstattuple</code> , <code>postgres_fdw</code> , <code>pg_trgm</code> |
| postgresql17-devel   | <code>pg_config</code> and headers — required to build any third-party extension                              |
| postgresql17-libs    | <code>libpq</code>                                                                                            |

```
# Where PGDG puts things (differs from the source layout above)
/usr/pgsql-17/bin          binaries
/var/lib/pgsql/17/data     PGDATA
/var/lib/pgsql/17/data/log logs
/usr/lib/systemd/system/postgresql-17.service
```

## 06 Installation — Part 2 — Extensions and the devel module

### CORE

**Extensions are how PostgreSQL grows without forking.** An extension is a packaged set of SQL objects plus, usually, a shared library. Installing the files makes it *available*; `CREATE EXTENSION` makes it *present in a database*. Those are two different steps and conflating them is the usual confusion.

```
-- What is available on this server?
SELECT name, default_version, installed_version, comment
FROM   pg_available_extensions ORDER BY name;

-- What is installed in THIS database?
SELECT extname, extversion, extnamespace::regnamespace FROM pg_extension;
\dx

CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION pgstattuple SCHEMA admin;
ALTER  EXTENSION pg_stat_statements UPDATE TO '1.11';
DROP   EXTENSION pgstattuple;
```

### The contrib extensions worth knowing by name

| EXTENSION                                         | USE                                                                             |
|---------------------------------------------------|---------------------------------------------------------------------------------|
| <code>pg_stat_statements</code>                   | Per-query execution statistics. <b>Install this on every production server.</b> |
| <code>pgstattuple</code>                          | Exact bloat measurement for tables and indexes                                  |
| <code>pg_buffercache</code>                       | What is actually in <code>shared_buffers</code> right now                       |
| <code>pg_trgm</code>                              | Trigram similarity; makes <code>LIKE '%x%'</code> indexable                     |
| <code>postgres_fdw</code> / <code>file_fdw</code> | Query another PostgreSQL server, or a flat file, as a table                     |
| <code>pgcrypto</code>                             | Hashing and encryption functions                                                |
| <code>btree_gin</code> / <code>btree_gist</code>  | Mixed-type composite indexes; exclusion constraints                             |
| <code>auto_explain</code>                         | Logs the plan of any statement over a duration threshold                        |
| <code>amcheck</code>                              | Verifies B-tree index structural integrity                                      |
| <code>pg_prewarm</code>                           | Reload the buffer cache after a restart                                         |

### Libraries that must be preloaded

Some extensions hook into the server before any database is open, and must be listed in `shared_preload_libraries` — which is a static parameter requiring a **restart**.

```
-- postgresql.conf
shared_preload_libraries = 'pg_stat_statements,auto_explain,pg_cron'
pg_stat_statements.max = 10000
pg_stat_statements.track = all
auto_explain.log_min_duration = '3s'
auto_explain.log_analyze = on
auto_explain.log_buffers = on
```

### The devel package and why it exists

`postgresql17-devel` provides `pg_config`, the server headers and the PGXS build infrastructure. Any extension not shipped as an RPM — `pg_repack`, `pgaudit`, `hypopg`, `pg_hint_plan`, TimescaleDB built from source — requires it.

```
sudo dnf install -y postgresql17-devel
export PATH=/usr/pgsql-17/bin:$PATH
pg_config --version && pg_config --pkglibdir && pg_config --sharedir

git clone https://github.com/reorg/pg_repack.git && cd pg_repack
make USE_PGXS=1
sudo make USE_PGXS=1 install
psql -d xxxx -c "CREATE EXTENSION pg_repack;"
```

#### VERSION DISCIPLINE

An extension's shared library is compiled against one major version's ABI. After a major upgrade, **every non-core extension must be reinstalled for the new version** before the cluster will start with it. Build an inventory of your extensions now — it becomes the critical checklist in Session 26.

## 07 Installation — Part 3 — Ubuntu, EDB, Percona Distribution

### CORE

#### Ubuntu and Debian — the PGDG APT repository

```
sudo apt install -y curl ca-certificates gnupg
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc \
  --fail https://www.postgresql.org/media/keys/ACCC4CF8.asc
echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc] \
  https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" \
  | sudo tee /etc/apt/sources.list.d/pgdg.list
sudo apt update
sudo apt install -y postgresql-17 postgresql-contrib-17 postgresql-server-dev-17
```

#### Debian's cluster model — genuinely different from Red Hat's

Debian and Ubuntu manage **multiple clusters per version** through a wrapper layer. This is a real operational difference, not a packaging detail.

```
pg_lsclusters
# Ver Cluster Port Status Owner    Data directory          Log file
# 17  main     5432 online postgres /var/lib/postgresql/17/main /var/log/postgresql/...

pg_createcluster 17 uat --port=5433 --start
pg_ctlcluster 17 main restart
pg_dropcluster 17 uat --stop
pg_upgradecluster 16 main          # wraps pg_upgrade
```

| ITEM              | RED HAT / PGDG RPM                  | DEBIAN / UBUNTU                                                |
|-------------------|-------------------------------------|----------------------------------------------------------------|
| Data directory    | <code>/var/lib/pgsql/17/data</code> | <code>/var/lib/postgresql/17/main</code>                       |
| Configuration     | Inside PGDATA                       | <code>/etc/postgresql/17/main/</code> — separate from the data |
| Logs              | <code>PGDATA/log/</code>            | <code>/var/log/postgresql/</code>                              |
| Service           | <code>postgresql-17</code>          | <code>postgresql@17-main</code>                                |
| Multiple clusters | Manual                              | First-class, via <code>pg_createcluster</code>                 |

#### EDB — EnterpriseDB

EDB ships two distinct things. **EDB Postgres Advanced Server** is a commercial distribution adding Oracle compatibility — PL/SQL syntax, packages, `DBMS_` equivalents, hierarchical queries, `ROWNUM`, an Oracle-compatible data dictionary. **EDB's community builds** are simply packaged PostgreSQL with an installer. Advanced Server is licensed; the community builds are not.

Relevant when migrating from Oracle: Advanced Server can absorb a large proportion of PL/SQL unchanged, which shortens a migration substantially — at the cost of a licence and of no longer being on plain PostgreSQL. Weigh that against the conversion effort discussed in Session 37.

#### Percona Distribution for PostgreSQL

Open source, no licence. Standard PostgreSQL plus a curated set of production extensions and tools pre-packaged and version-tested together.

```
sudo dnf install -y https://repo.percona.com/yum/percona-release-latest.noarch.rpm
sudo percona-release setup ppg-17
sudo dnf install -y percona-postgresql17-server percona-postgresql17-contrib
```

Bundled: `pg_repack`, `pgaudit`, `pgBackRest`, `pg_stat_monitor`, `patroni`, `pgbouncer`, `wal2json`, HAProxy and etcd. The appeal is that the whole HA and backup stack arrives from one repository, already tested against one another.

| DISTRIBUTION                | CHOOSE WHEN                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------|
| <b>PGDG</b>                 | Default. Closest to upstream, fastest security updates, largest community.        |
| <b>Percona</b>              | You want the HA and backup stack packaged and version-matched, without a licence. |
| <b>EDB Advanced Server</b>  | Oracle compatibility materially reduces a migration, and a licence is acceptable. |
| <b>Distribution default</b> | Rarely — usually an older minor version on a slower update cycle.                 |



## 08 Installation — Part 4 — Scenarios and troubleshooting

### CORE

#### Multiple versions and clusters on one host

```
# Red Hat: two majors side by side, different ports
sudo dnf install -y postgresql16-server postgresql17-server
sudo PGSETUP_INITDB_OPTIONS="--data-checksums" /usr/pgsql-16/bin/postgresql-16-setup initdb
sudo PGSETUP_INITDB_OPTIONS="--data-checksums" /usr/pgsql-17/bin/postgresql-17-setup initdb
sudo -u postgres sed -i "s/^#port = 5432/port = 5433/" /var/lib/pgsql/17/data/postgresql.conf
sudo systemctl enable --now postgresql-16 postgresql-17

# A second cluster of the SAME version
sudo -u postgres /usr/pgsql-17/bin/initdb -D /data/pg17_uat --data-checksums
sudo -u postgres /usr/pgsql-17/bin/pg_ctl -D /data/pg17_uat -o "-p 5434" -l /data/pg17_uat/uat.log start
```

#### Installation failures and what they mean

| ERROR                                                                      | CAUSE AND FIX                                                                                                                                     |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| initdb: cannot be run as root                                              | Run as the <code>postgres</code> user. The server refuses to run as root by design.                                                               |
| directory "/data/pg" exists but is not empty                               | <code>initdb</code> requires an empty directory. Clear it or choose another.                                                                      |
| could not create shared memory segment                                     | Kernel limits. Modern PostgreSQL uses <code>mmap</code> so this is rarer; check <code>vm.overcommit_memory</code> and <code>cgroup</code> limits. |
| FATAL: could not create lock file "/var/run/postgresql/.s.PGSQL.5432.lock" | Socket directory missing or wrong ownership after a reboot. Use <code>systemd-tmpfiles</code> or create it in the unit.                           |
| Permission denied on PGDATA                                                | Must be owned by <code>postgres</code> and mode <b>0700</b> (0750 is accepted from PG 11). The server refuses looser permissions.                 |
| could not bind IPv4 address: Address already in use                        | Another cluster on that port. <code>ss -lntp   grep 5432</code> .                                                                                 |
| database files are incompatible with server                                | Wrong major-version binaries against this PGDATA. Check <code>cat \$PGDATA/PG_VERSION</code> .                                                    |
| pg_config: command not found                                               | <code>-devel</code> package missing, or <code>/usr/pgsql-17/bin</code> not on PATH.                                                               |
| SELinux denials on a custom data directory                                 | <code>semanage fcontext -a -t postgresql_db_t "/data/pgsql17(/.*)?" &amp;&amp; restorecon -Rv /data/pgsql17</code>                                |

#### Post-installation verification — run these before handing the server over

```
psql -c "SELECT version();"
psql -c "SHOW data_directory; SHOW config_file; SHOW hba_file;"
psql -c "SELECT name, setting
        WHERE name IN ('data_checksums', 'wal_level', 'max_connections',
                      'shared_buffers', 'listen_addresses', 'password_encryption');"
psql -c "SELECT datname, pg_encoding_to_char(encoding), datcollate, datctype FROM pg_database;"
pg_controldata $PGDATA | head -25
```

## OS-level preparation that is part of installation, not tuning

```
# Huge pages – reduces page-table overhead at large shared_buffers
# size = shared_buffers / 2MB, plus headroom
sysctl -w vm.nr_hugepages=4200
# postgresql.conf: huge_pages = try

# Avoid the OOM killer choosing the postmaster
echo -1000 > /proc/$(head -1 $PGDATA/postmaster.pid)/oom_score_adj

vm.swappiness = 1
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
# Disable transparent huge pages – known to cause latency spikes
echo never > /sys/kernel/mm/transparent_hugepage/enabled
# Scheduler for SSD/NVMe
echo none > /sys/block/nvme0n1/queue/scheduler
```

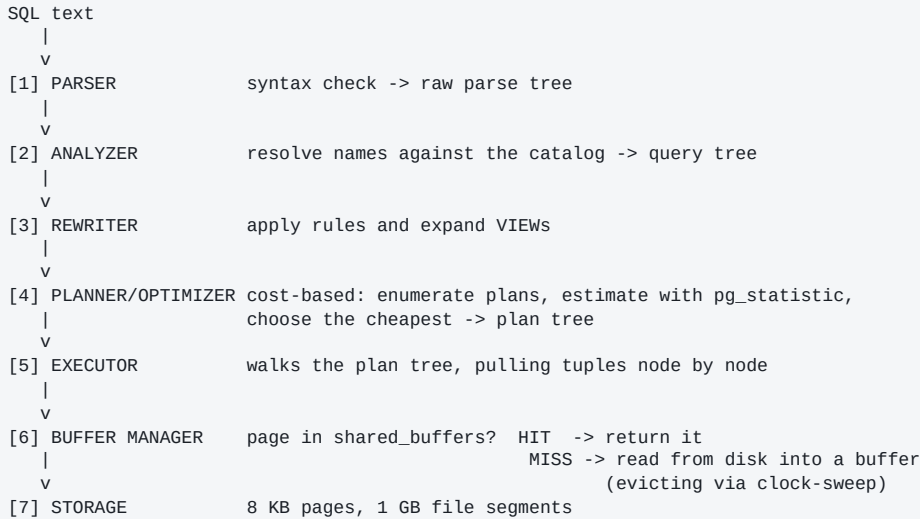
### FILESYSTEM CHOICE

XFS or ext4 are the tested options. Mount with `noatime`. Keep `pg_wal` on a separate filesystem — not only for I/O separation, but because it means a WAL runaway fills its own volume and not the one holding your data, which turns a full outage into a degraded one.

## 09 Architecture — The CRUD Workflow end to end

### CORE

What happens between your **SELECT** and the answer. Every performance topic later in the course — planner settings, shared\_buffers, checkpoints, WAL — is a lever on one of these stages.



### The write path, which is where the interesting behaviour lives

```
UPDATE xxxx_orders SET status='SHIPPED' WHERE id=100;
```

1. Locate the page (buffer hit or disk read)
2. Write a WAL record describing the change -> WAL buffer
3. Insert a NEW tuple version in the page; set xmax on the old one
4. Mark the buffer DIRTY
5. COMMIT: flush WAL to disk (fsync) <-- durability happens HERE
6. Return success to the client

The DATA PAGE is still only in memory. It reaches disk later, via:

- the background writer, trickling dirty buffers out, or
- the checkpoint, flushing all dirty buffers at a checkpoint, or
- a backend needing a free buffer and having to evict a dirty one

#### THE INSIGHT THAT EXPLAINS CHECKPOINT TUNING

Commit durability costs **one sequential WAL write**. Data pages are written later, in bulk, by the checkpoint. That is why PostgreSQL sustains high write rates — and why an aggressive checkpoint produces a burst of random I/O. Sessions 31 and 32 tune exactly this trade-off.

### HOT updates — why some updates are far cheaper

If an update changes **no indexed column** and the new version fits on the **same page**, PostgreSQL performs a Heap-Only Tuple update: the new version is chained from the old within the page and **no index entry is written**. Index maintenance is the expensive part of an update, so HOT is a large saving.

```

SELECT relname, n_tup_upd, n_tup_hot_upd,
       round(100.0*n_tup_hot_upd/NULLIF(n_tup_upd,0),1) AS hot_pct
FROM   pg_stat_user_tables WHERE n_tup_upd > 0 ORDER BY n_tup_upd DESC LIMIT 10;

-- Leave room on each page for HOT updates on a heavily updated table
ALTER TABLE xxxx_orders SET (fillfactor = 85);

```

A low **hot\_pct** on a hot table means either the updates touch indexed columns, or pages are too full. **fillfactor** addresses the second.

## Reading a plan against this model

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT o.*, c.name FROM xxxx_orders o JOIN xxxx_customers c USING (customer_id)
WHERE o.status = 'PENDING';

-- shared hit=N    pages served from shared_buffers      (stage 6, hit)
-- shared read=N   pages read from disk or OS cache       (stage 6, miss)
-- rows estimated vs actual -> quality of stage 4's estimate
```

## 10 Logger Process — How PostgreSQL logging works

### CORE

The log is your only record of what happened when nobody was watching. Sessions 33 to 35 analyse it with pgBadger; this session makes sure it contains something worth analysing.

### How it works

Backends write log messages to stderr. If `logging_collector` is on, a dedicated **logger process** captures that stream and writes it to files, handling rotation. The logger is deliberately simple and restartable so that a logging failure cannot take down the server.

### The configuration that matters

```
logging_collector = on
log_destination = 'stderr'          -- or 'csvlog', or 'jsonlog' (PG 15+)
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_rotation_age = '1d'
log_rotation_size = '100MB'
log_truncate_on_rotation = on

-- WHAT to log
log_min_duration_statement = 1000    -- every statement over 1s. The single best setting.
log_min_messages = warning
log_min_error_statement = error
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on                 -- who blocked whom, after deadlock_timeout
log_temp_files = 10MB               -- work_mem spills
log_autovacuum_min_duration = 1000
log_statement = 'ddl'               -- none | ddl | mod | all
log_error_verbosity = default

-- The line prefix. pgBadger REQUIRES this exact shape.
log_line_prefix = '%m [%p] %q%u@%d %a %h '
```

| ESCAPE | MEANING                             | ESCAPE | MEANING             |
|--------|-------------------------------------|--------|---------------------|
| %m     | Timestamp with milliseconds         | %u     | User name           |
| %p     | Process ID                          | %d     | Database name       |
| %a     | Application name                    | %h     | Client host         |
| %q     | Stop here for non-session processes | %l     | Session line number |

### Structured output

```
-- CSV: loadable into a table for SQL analysis
log_destination = 'csvlog'
CREATE TABLE postgres_log (...);      -- schema in the PostgreSQL docs
COPY postgres_log FROM '/var/lib/pgsql/17/data/log/postgresql.csv' WITH csv;

-- JSON (PG 15+): for Loki, Elasticsearch, CloudWatch
log_destination = 'jsonlog'
```

**TWO SETTINGS TO WEIGH CAREFULLY**

`log_statement = 'all'` logs every statement including its parameters. On a busy system that is an enormous volume, a performance cost, and a **compliance exposure** — personal data ends up in plain text in the log. Use `log_min_duration_statement` instead for performance work, and `log_statement = 'ddl'` for an audit trail of schema change. If you need full statement auditing, use the `pgaudit` extension, which is designed for it.

**Checking that logging is working**

```
SELECT pg_current_logfile();
SELECT name, setting FROM pg_settings WHERE name LIKE 'log%' AND setting <> 'off';

-- Rotate on demand
SELECT pg_rotate_logfile();

-- Quick triage from the shell
grep -c "ERROR"      $PGDATA/log/postgresql-*.log
grep -c "FATAL"      $PGDATA/log/postgresql-*.log
grep "deadlock"      $PGDATA/log/postgresql-*.log
grep "checkpoint complete" $PGDATA/log/postgresql-*.log | tail -20
```

**SET THESE ON DAY ONE**

`log_min_duration_statement`, `log_checkpoints`, `log_lock_waits`, `log_temp_files` and `log_autovacuum_min_duration`. Together they cost almost nothing and they are the difference between diagnosing yesterday's incident and guessing about it.

## 11 Autovacuum Part 1 — What it does and why it matters

### CORE

**The problem it solves.** MVCC leaves dead tuples behind. Nothing reclaims them automatically at the storage layer, so without a cleanup process every table grows forever, every index degrades, and eventually transaction IDs wrap around and the cluster refuses to accept writes.

### Five jobs, not one

| JOB                          | CONSEQUENCE IF IT DOES NOT HAPPEN                                     |
|------------------------------|-----------------------------------------------------------------------|
| Reclaim dead tuples          | Tables and indexes bloat; sequential scans read mostly garbage        |
| Update the visibility map    | Index Only Scans become impossible; queries fall back to heap fetches |
| Freeze old transaction IDs   | Transaction ID wraparound; the cluster eventually stops               |
| Update the free space map    | New rows extend the file instead of reusing space                     |
| Refresh statistics (ANALYZE) | The planner estimates badly and chooses wrong plans                   |

### How a table is chosen

```
vacuum threshold = autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples
                  = 50                          + 0.2                          * reltuples

analyze threshold = autovacuum_analyze_threshold + autovacuum_analyze_scale_factor * reltuples
                   = 50                          + 0.1                          * reltuples

-- PG 13+ also triggers on inserts alone, which matters for append-only tables:
insert threshold  = autovacuum_vacuum_insert_threshold + insert_scale_factor * reltuples
                   = 1000                          + 0.2                          * reltuples
```

### The launcher and the workers

```
autovacuum launcher (one per cluster)
  | wakes every autovacuum_naptime (default 1 min)
  | picks a database, checks pg_stat_all_tables against the thresholds
  v
autovacuum worker (up to autovacuum_max_workers, default 3)
  | vacuums one table at a time
  | throttled by the cost-delay mechanism so it does not swamp I/O
  v
dead tuples reclaimed, visibility map updated, statistics refreshed
```

### Watching it work

```
-- Currently running
SELECT p.pid, p.datname, p.relid::regclass AS table, p.phase,
       p.heap_blks_total, p.heap_blks_scanned, p.heap_blks_vacuumed,
       round(100.0*p.heap_blks_scanned/NULLIF(p.heap_blks_total,0),1) AS pct,
       now() - a.query_start AS running_for
FROM   pg_stat_progress_vacuum p JOIN pg_stat_activity a USING (pid);

-- History per table
SELECT relname, n_live_tup, n_dead_tup,
       vacuum_count, autovacuum_count, analyze_count, autoanalyze_count,
       last_vacuum, last_autovacuum, last_analyze, last_autoanalyze
FROM   pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 15;
```

**NEVER DISABLE AUTOVACUUM**

It is occasionally suggested as a fix for I/O pressure. It is not a fix; it converts a throughput problem into an outage several weeks later. If autovacuum is costing too much I/O, **tune the cost limits** so it works more steadily, or fix whatever is preventing it from completing. Session 12 covers both.



## 12 Autovacuum Part 2 — Tuning the parameters

CORE

The parameters, grouped by what they control

| PARAMETER                                    | DEFAULT   | EFFECT                                                          |
|----------------------------------------------|-----------|-----------------------------------------------------------------|
| <code>autovacuum</code>                      | on        | Master switch. Leave on.                                        |
| <code>autovacuum_max_workers</code>          | 3         | Concurrent workers. <b>Restart required.</b>                    |
| <code>autovacuum_naptime</code>              | 1min      | How often the launcher wakes                                    |
| <code>autovacuum_vacuum_scale_factor</code>  | 0.2       | <b>20% dead before acting</b> — far too high for large tables   |
| <code>autovacuum_vacuum_threshold</code>     | 50        | Flat addition to the above                                      |
| <code>autovacuum_analyze_scale_factor</code> | 0.1       | Same idea for statistics                                        |
| <code>autovacuum_vacuum_cost_limit</code>    | -1 (=200) | Work budget before sleeping. <b>Raise this to go faster.</b>    |
| <code>autovacuum_vacuum_cost_delay</code>    | 2ms       | Sleep duration once the budget is spent                         |
| <code>autovacuum_freeze_max_age</code>       | 200M      | Forces anti-wraparound vacuum, even if autovacuum is off        |
| <code>maintenance_work_mem</code>            | 64MB      | Memory per worker for dead-tuple tracking. <b>Raise to 1GB.</b> |
| <code>vacuum_cost_page_hit/miss/dirty</code> | 1/2/20    | The cost accounting behind the budget                           |

A production starting point

```
-- Cluster-wide
ALTER SYSTEM SET autovacuum_max_workers = 6;           -- restart
ALTER SYSTEM SET autovacuum_naptime = '15s';
ALTER SYSTEM SET autovacuum_vacuum_cost_limit = 2000;  -- 10x the default
ALTER SYSTEM SET autovacuum_vacuum_cost_delay = '2ms';
ALTER SYSTEM SET maintenance_work_mem = '1GB';
ALTER SYSTEM SET autovacuum_vacuum_scale_factor = 0.05;
ALTER SYSTEM SET autovacuum_analyze_scale_factor = 0.02;
ALTER SYSTEM SET log_autovacuum_min_duration = '1s';
SELECT pg_reload_conf();
```

Per-table overrides — where the real work is

Cluster-wide settings are a compromise. Large or hot tables need their own.

```
-- A very large, heavily updated table
ALTER TABLE xxxx_orders SET (
    autovacuum_vacuum_scale_factor = 0.01,      -- 1% instead of 20%
    autovacuum_vacuum_threshold    = 5000,
    autovacuum_analyze_scale_factor = 0.005,
    autovacuum_vacuum_cost_limit    = 4000,      -- let it work hard on this one
    fillfactor                      = 85          -- room for HOT updates
);

-- A small, extremely hot table (a queue or counter)
ALTER TABLE xxxx_job_queue SET (
    autovacuum_vacuum_scale_factor = 0.0,
    autovacuum_vacuum_threshold    = 100,        -- vacuum after only 100 dead rows
    autovacuum_naptime             = '10s'
);

-- Inspect current overrides
SELECT relname, reloptions FROM pg_class WHERE reloptions IS NOT NULL;
-- Remove one
ALTER TABLE xxxx_orders RESET (autovacuum_vacuum_scale_factor);
```

### The cost model, stated plainly

Each page the worker touches adds to a running cost:

|                              |    |                        |      |
|------------------------------|----|------------------------|------|
| page found in shared_buffers | -> | vacuum_cost_page_hit   | (1)  |
| page read from disk          | -> | vacuum_cost_page_miss  | (2)  |
| page modified by the vacuum  | -> | vacuum_cost_page_dirty | (20) |

When the total reaches `autovacuum_vacuum_cost_limit`, the worker sleeps for `autovacuum_vacuum_cost_delay`, then resumes.

Raising the LIMIT or lowering the DELAY makes autovacuum faster and noisier. On modern SSD storage the defaults are extremely conservative.

#### THE SIZING QUESTION TO ASK

“How long does one full pass over my largest table take at the current cost settings, and is that shorter than the interval at which it accumulates 20% dead rows?” If not, autovacuum can never catch up and bloat grows monotonically, no matter how healthy the logs look.

## 13 Autovacuum Part 3 — Bloat detection and remediation

### CORE

#### Detecting bloat, three levels of rigour

```
-- LEVEL 1: free, instant, approximate
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/NULLIF(n_live_tup+n_dead_tup,0),1) AS dead_pct,
       last_autovacuum,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 1000
ORDER BY n_dead_tup DESC LIMIT 20;
```

```
-- LEVEL 2: exact, but scans the relation — one table at a time
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple('xxxx_orders');
--  table_len | tuple_count | tuple_percent | dead_tuple_count
--          | dead_tuple_percent | free_space | free_percent

SELECT * FROM pgstatindex('idx_xxxx_orders_created');
--  leaf_fragmentation, avg_leaf_density  -> index health

-- Cheaper approximation on very large tables (PG 12+)
SELECT * FROM pgstattuple_approx('xxxx_orders');
```

```
-- LEVEL 3: estimate across the whole database with no table scan —
--          use the community postgresql-bloat-estimation queries, or:
SELECT schemaname, relname,
       pg_size_pretty(pg_relation_size(relid))          AS actual,
       pg_size_pretty((pg_relation_size(relid)
       * n_dead_tup::numeric / NULLIF(n_live_tup+n_dead_tup,0))::bigint) AS est_bloat
FROM   pg_stat_user_tables
WHERE  n_live_tup + n_dead_tup > 0
ORDER BY pg_relation_size(relid) DESC LIMIT 20;
```

#### Interpreting the number

| BLOAT     | VERDICT                                                                             |
|-----------|-------------------------------------------------------------------------------------|
| Under 20% | <b>Normal and healthy.</b> That free space is reused. Do nothing.                   |
| 20–40%    | Watch. Review the table's autovacuum settings.                                      |
| 40–100%   | Act. Usually index bloat first — try <code>REINDEX CONCURRENTLY</code> .            |
| Over 100% | Autovacuum is being prevented from working. <b>Find the cause before repacking.</b> |

#### Why VACUUM runs but frees nothing — the question that separates DBAs

If `last_autovacuum` is recent and `n_dead_tup` still climbs, VACUUM is executing and reclaiming nothing. A dead tuple can only be removed if it is invisible to **every** snapshot in the cluster. Something is holding the xmin horizon back.

```
-- Suspect 1: a long-running transaction, anywhere in the cluster
SELECT pid, datname, username, state, now()-xact_start AS xact_age,
       age(backend_xmin) AS xmin_age, left(query,60)
FROM   pg_stat_activity WHERE backend_xmin IS NOT NULL
ORDER  BY age(backend_xmin) DESC LIMIT 5;

-- Suspect 2: a replication slot (logical slots also hold catalog_xmin)
SELECT slot_name, active, wal_status, age(xmin), age(catalog_xmin)
FROM   pg_replication_slots;

-- Suspect 3: an orphaned prepared transaction – invisible in pg_stat_activity
SELECT gid, prepared, owner, database FROM pg_prepared_xacts;

-- Suspect 4: hot_standby_feedback from a standby running long queries
SHOW hot_standby_feedback;  -- on the STANDBY
```

Remediation

| ACTION                     | AVAILABILITY         | NOTES                                                  |
|----------------------------|----------------------|--------------------------------------------------------|
| VACUUM (VERBOSE, ANALYZE)  | No impact            | Always try first                                       |
| REINDEX INDEX CONCURRENTLY | No impact (PG 12+)   | Index bloat is often the whole problem                 |
| pg_repack                  | Brief lock at swap   | <b>The production answer.</b> Needs 2× table size free |
| VACUUM FULL                | <b>Table offline</b> | Maintenance window only                                |
| CLUSTER                    | <b>Table offline</b> | Also physically orders by an index                     |
| Drop a partition           | Brief                | Instant, complete, no VACUUM needed                    |

```
pg_repack -d xxxx -t public.xxxx_orders --jobs 4
pg_repack -d xxxx --only-indexes -t public.xxxx_orders
REINDEX TABLE CONCURRENTLY xxxx_orders;
```

THE DESIGN ANSWER

A table that needs repacking every quarter is telling you it should be partitioned on whatever key you purge by. `DROP TABLE xxxx_orders_2024_q1` is instantaneous, returns every byte, and needs no maintenance window.

## 14 Post-Installation — pg\_hba.conf and SCRAM-SHA-256

### CORE

A freshly initialised cluster is not production-ready. This session is the hardening checklist that turns it into one.

#### pg\_hba.conf structure

| # | TYPE    | DATABASE    | USER          | ADDRESS        | METHOD        |
|---|---------|-------------|---------------|----------------|---------------|
|   | local   | all         | postgres      |                | peer          |
|   | local   | all         | all           |                | scram-sha-256 |
|   | hostssl | xxxx        | xxxx_app      | 10.20.30.0/24  | scram-sha-256 |
|   | hostssl | xxxx        | +xxxx_readers | 10.20.40.0/24  | scram-sha-256 |
|   | host    | replication | replicator    | 10.20.30.41/32 | scram-sha-256 |
|   | hostssl | all         | all           | 0.0.0.0/0      | reject        |

| FIELD    | VALUES WORTH KNOWING                                                          |
|----------|-------------------------------------------------------------------------------|
| TYPE     | local (Unix socket), host (TCP either way), hostssl (TLS required), hostnossl |
| DATABASE | A name, a list, all, replication, or @filename                                |
| USER     | A role, all, or +groupname to match members of a group                        |
| ADDRESS  | CIDR, hostname, samehost, samenet                                             |
| METHOD   | scram-sha-256, md5, cert, ldap, gss, peer, trust, reject                      |

#### Migrating to SCRAM-SHA-256 — order matters

```
-- STEP 1: change the algorithm used for NEW password hashes
ALTER SYSTEM SET password_encryption = 'scram-sha-256';
SELECT pg_reload_conf();
SHOW password_encryption;

-- STEP 2: find every role still holding an md5 hash
SELECT rolname,
       CASE WHEN rolpassword LIKE 'SCRAM-SHA-256%' THEN 'scram'
            WHEN rolpassword LIKE 'md5%' THEN 'md5 -- must be reset'
            ELSE 'none' END AS hash_type
FROM   pg_authid WHERE rolcanlogin ORDER BY 2;

-- STEP 3: reset each one. This re-hashes with the new algorithm.
ALTER ROLE xxxx_app PASSWORD 'new-strong-password';

-- STEP 4: only NOW change pg_hba.conf lines to scram-sha-256, and reload.
```

#### REVERSE THIS ORDER AND YOU LOCK EVERYONE OUT

A `scram-sha-256` rule cannot authenticate a role whose stored hash is still md5. Change the algorithm, reset every password, verify, then change the rules.

## TLS

```
-- postgresql.conf
ssl = on
ssl_cert_file = '/etc/pki/postgres/server.crt'
ssl_key_file = '/etc/pki/postgres/server.key'      -- chmod 600, owned by postgres
ssl_ca_file = '/etc/pki/postgres/root.crt'
ssl_min_protocol_version = 'TLSv1.2'
ssl_prefer_server_ciphers = on

-- Verify per session
SELECT pid, username, ssl, version, cipher, client_addr
FROM   pg_stat_ssl JOIN pg_stat_activity USING (pid);
```

## The rest of the hardening checklist

```
-- Least privilege on the public schema
REVOKE ALL ON DATABASE xxxx FROM PUBLIC;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;      -- default only from PG 15

-- Separate ownership from application access
CREATE ROLE xxxx_owner NOLOGIN;
CREATE ROLE xxxx_app LOGIN PASSWORD '...' CONNECTION LIMIT 50 IN ROLE xxxx_rw;
ALTER DATABASE xxxx OWNER TO xxxx_owner;

-- Safety timeouts
ALTER SYSTEM SET idle_in_transaction_session_timeout = '10min';
ALTER SYSTEM SET statement_timeout = 0;          -- set per role instead
ALTER SYSTEM SET lock_timeout = '10s';
ALTER ROLE xxxx_report SET statement_timeout = '5min';

-- Verify the ruleset is valid and in the order you intended
SELECT line_number, type, database, user_name, address, auth_method, error
FROM   pg_hba_file_rules ORDER BY line_number;
```

## 15 psql — Command-line mastery for DBAs

### CORE

**psql** is the DBA's primary interface. Fluency here is worth more day to day than any GUI.

#### Inspection meta-commands

| COMMAND                | SHOWS                      | COMMAND                | SHOWS                         |
|------------------------|----------------------------|------------------------|-------------------------------|
| <code>\l+</code>       | Databases with sizes       | <code>\dt+</code>      | Tables with sizes             |
| <code>\dn+</code>      | Schemas and privileges     | <code>\di+</code>      | Indexes                       |
| <code>\d+ tbl</code>   | Full table definition      | <code>\dp tbl</code>   | Object privileges             |
| <code>\du+</code>      | Roles and attributes       | <code>\df+</code>      | Functions                     |
| <code>\dv / \dm</code> | Views / materialised views | <code>\dx</code>       | Installed extensions          |
| <code>\ds</code>       | Sequences                  | <code>\dconfig</code>  | Non-default settings (PG 15+) |
| <code>\dP</code>       | Partitioned relations      | <code>\conninfo</code> | Current connection details    |

#### Formatting and workflow

```

\X auto          -- expanded output when a row is too wide. Use this always.
\timing on       -- report execution time for every statement
\pset pager off
\pset null '(null)'
\e              -- edit the last query in $EDITOR
\watch 2         -- re-run the last query every 2 seconds
\i script.sql    -- run a file
\o out.txt       -- redirect output to a file
\! ls -la        -- shell out
\copy (SELECT * FROM xxxx_orders) TO 'out.csv' WITH (FORMAT csv, HEADER)
\g              -- re-execute
\gx             -- execute with expanded output
\s              -- command history

```

#### Variables and dynamic SQL

```

\set schema_name 'public'
\set ON_ERROR_STOP on
SELECT :'schema_name';          -- quoted as a literal
SELECT * FROM :"schema_name".xxxx_orders; -- quoted as an identifier

-- \gexec runs the RESULT of a query as SQL. Extremely useful for bulk DDL.
SELECT format('REINDEX INDEX CONCURRENTLY %I.%I;', schemaname, indexrelname)
FROM   pg_stat_user_indexes WHERE idx_scan = 0
\gexec

```

## A .psqlrc worth having

```
-- ~/.psqlrc
\set QUIET 1
\timing on
\x auto
\set ON_ERROR_ROLLBACK interactive
\set COMP_KEYWORD_CASE upper
\set HISTFILE ~/.psql_history- :DBNAME
\set HISTSIZE 5000
\set PROMPT1 '%[%033[1;32m%]n@%/%[%033[0m%]R%# '
\pset null '(null)'
\set QUIET 0

-- Shortcuts you will use constantly
\set activity 'SELECT pid, state, wait_event_type, wait_event, now()-query_start AS runtime, left(query,60) FROM
pg_stat_activity WHERE state <> ''idle'' ORDER BY query_start;'
\set blocking 'SELECT pid, pg_blocking_pids(pid), left(query,60) FROM pg_stat_activity WHERE
cardinality(pg_blocking_pids(pid)) > 0;'
\set slow 'SELECT round(total_exec_time::numeric) AS ms, calls, left(query,60) FROM pg_stat_statements ORDER
BY total_exec_time DESC LIMIT 10;'
-- then just type: :activity
```

## Scripting safely

```
# The flags that matter in automation
psql -v ON_ERROR_STOP=1 --single-transaction -f migration.sql -d xxxx
# ON_ERROR_STOP=1 : exit non-zero on the first error — WITHOUT THIS A FAILED
# MIGRATION SCRIPT STILL EXITS 0
# --single-transaction: all or nothing
psql -qAtX -c "SELECT count(*) FROM xxxx_orders" # clean value for a shell variable
# -q quiet -A unaligned -t tuples only -X ignore .psqlrc
```

### THE FLAG THAT PREVENTS SILENT FAILURE

Without `-v ON_ERROR_STOP=1`, psql continues after an error and exits 0. Deployment pipelines report success while half the migration did not apply. Put it in every scripted invocation.



## 16 WAL Files — Write-Ahead Logging internals and management

CORE

### Structure

| CONCEPT         | DETAIL                                                                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LSN</b>      | Log Sequence Number — a byte offset in the WAL stream, printed as <code>3/17000028</code> . Every position in replication, backup and recovery is an LSN. |
| <b>Segment</b>  | A 16 MB file by default (settable at <code>initdb</code> with <code>--wal-segsize</code> )                                                                |
| <b>Naming</b>   | 24 hex characters: timeline (8) + logical id (8) + segment (8), e.g. <code>0000000100000000300000017</code>                                               |
| <b>Timeline</b> | Increments on every promotion. Prevents a promoted standby's WAL from being confused with the old primary's.                                              |
| <b>Record</b>   | A description of one change: which relation, which block, what changed                                                                                    |

```
SELECT pg_current_wal_lsn(), pg_current_wal_insert_lsn();
SELECT pg_walfile_name(pg_current_wal_lsn());
SELECT pg_size_pretty(pg_wal_lsn_diff('3/18000000', '3/17000000')) AS bytes_between;
SELECT * FROM pg_ls_waldir() ORDER BY modification DESC LIMIT 10;
```

### The parameters

| PARAMETER                                                | TYPICAL   | EFFECT                                                                                 |
|----------------------------------------------------------|-----------|----------------------------------------------------------------------------------------|
| <code>wal_level</code>                                   | replica   | <code>minimal</code> / <code>replica</code> / <code>logical</code> . Restart required. |
| <code>max_wal_size</code>                                | 4–16GB    | Soft target that triggers a checkpoint                                                 |
| <code>min_wal_size</code>                                | 2GB       | Segments kept for recycling rather than deletion                                       |
| <code>wal_buffers</code>                                 | -1 (auto) | Auto is 1/32 of <code>shared_buffers</code> , capped at 16 MB                          |
| <code>wal_compression</code>                             | on        | Compresses full-page images. Large saving on replication bandwidth.                    |
| <code>full_page_writes</code>                            | on        | <b>Never turn off</b> unless the storage guarantees atomic 8 KB writes                 |
| <code>wal_keep_size</code>                               | 1–4GB     | Retention floor when no slot is used                                                   |
| <code>archive_mode</code> / <code>archive_command</code> | on        | Required for PITR                                                                      |
| <code>checkpoint_timeout</code>                          | 15min     | Time-based checkpoint interval                                                         |
| <code>checkpoint_completion_target</code>                | 0.9       | Spreads checkpoint writes over the interval                                            |

### Full-page writes, and why they exist

The first time a page is modified after a checkpoint, PostgreSQL writes the **entire 8 KB page** into WAL, not just the change. This protects against torn pages — a partial write when the OS page size and the storage sector size disagree during a crash. It is also why WAL volume spikes immediately after each checkpoint, and why lengthening `checkpoint_timeout` reduces total WAL written.

## Checkpoints

```
-- Are checkpoints being forced rather than scheduled?
SELECT num_timed, num_requested,
       round(100.0*num_requested/NULLIF(num_timed+num_requested,0),1) AS forced_pct,
       write_time, sync_time, buffers_written
FROM   pg_stat_checkpointer;          -- PG16+; earlier: pg_stat_bgwriter
```

`forced_pct` above roughly 10% means `max_wal_size` is too small: the server is checkpointing because WAL filled, not because time elapsed. Raise `max_wal_size` — it costs disk, and it buys both fewer checkpoints and less WAL from full-page writes.

## Managing the directory

```
-- How much WAL is present, and how much is waiting to be archived?
SELECT count(*) AS segments, pg_size_pretty(sum(size)) AS total FROM pg_ls_waldir();
SELECT count(*) FROM pg_ls_dir('pg_wal/archive_status') f WHERE f LIKE '%.ready';

-- Who is retaining it?
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM   pg_replication_slots ORDER BY 4 DESC;

-- Force a segment switch (useful before a backup boundary)
SELECT pg_switch_wal();
```

### NEVER DELETE FROM PG\_WAL BY HAND

Those segments may be unarchived or needed for recovery. Deleting them corrupts the cluster and invalidates every backup that depended on them. Fix the cause — a dead slot or a failing `archive_command` — and PostgreSQL recycles them itself at the next checkpoint.

## 17 Datafiles and Control Files — Physical structure

### CORE

#### The data directory

```
$PGDATA/
PG_VERSION          major version, as text
postgresql.conf      main configuration
postgresql.auto.conf written by ALTER SYSTEM — overrides the above
pg_hba.conf          host-based authentication
pg_ident.conf        OS-user to DB-role mapping
postmaster.pid       PID, data directory, port, socket dir, shmkey
global/
  pg_control         THE control file
base/
  16384/              one directory per database, named by OID
                     a database
                     2601, 2601_fsm, 2601_vm, 2601.1 ...
pg_wal/
  archive_status/    .ready and .done markers
pg_xact/
  pg_multixact/      multi-transaction status (row-level shared locks)
pg_subtrans/         subtransaction status
pg_tblspc/           symlinks to tablespaces
pg_stat/ pg_stat_tmp/ statistics
pg_logical/ pg_replslot/ logical decoding and replication slots
log/                 server log (Red Hat layout)
```

#### How a table maps to files

```
SELECT oid AS db_oid, datname FROM pg_database WHERE datname = 'xxxx';
SELECT relname, relfilenode, pg_relation_filepath('xxxx_orders') FROM pg_class
WHERE relname = 'xxxx_orders';
-- base/16384/24576
```

| FILE        | FORK                                                                                          |
|-------------|-----------------------------------------------------------------------------------------------|
| 24576       | Main fork — the heap itself                                                                   |
| 24576.1, .2 | Additional 1 GB segments as the relation grows                                                |
| 24576_fsm   | Free Space Map — which pages have room                                                        |
| 24576_vm    | Visibility Map — which pages are all-visible and all-frozen. <b>Enables Index Only Scans.</b> |
| 24576_init  | Initialisation fork, for unlogged relations                                                   |

#### RELFILENODE IS NOT THE OID, AND IT CHANGES

`VACUUM FULL`, `CLUSTER`, `REINDEX` and `TRUNCATE` all rewrite the relation and assign a **new relfilenode**. Never build tooling that assumes the filename is stable — always resolve it through `pg_relation_filepath()`.

## Page layout

```

One 8 KB page:
+-----+
| PageHeaderData (24 bytes): LSN, checksum, flags, pointers |
+-----+
| ItemIdData array (line pointers) --> |
+-----+
|               free space               |
+-----+
|               <-- tuples, filled from the end             |
+-----+
| special space (index access methods only) |
+-----+

ctid = (page number, line pointer index)

```

## TOAST

A tuple cannot span pages, so values over roughly 2 KB are compressed and, if still too large, sliced into chunks in a side table. This is transparent, but it means a table's real size includes its TOAST relation.

```

SELECT c.relname, t.relname AS toast_table,
       pg_size_pretty(pg_relation_size(c.oid)) AS main,
       pg_size_pretty(pg_relation_size(t.oid)) AS toast
FROM   pg_class c LEFT JOIN pg_class t ON t.oid = c.reltoastrelid
WHERE  c.relname = 'xxxx_orders';

ALTER TABLE xxxx_orders ALTER COLUMN payload SET STORAGE EXTERNAL; -- no compression
-- PLAIN | EXTENDED (default) | EXTERNAL | MAIN

```

## The control file

```

pg_controldata $PGDATA
# Database cluster state:           in production
# Latest checkpoint location:       3/17000060
# Latest checkpoint's REDO location: 3/17000028
# Latest checkpoint's NextXID:      0:89234
# Time of latest checkpoint:        ...
# wal_level setting:                replica
# Data page checksum version:       1
# Database block size:              8192

```

`pg_control` is 8 KB and holds the cluster's identity and recovery starting point. **If it is lost or corrupt the cluster will not start.** `pg_resetwal` can fabricate a new one, but it discards WAL and produces an inconsistent database — it is a last resort before a restore from backup, never a routine tool.

## 18 Shutdown Modes — Smart, fast, immediate

### CORE

| MODE                                             | SIGNAL               | BEHAVIOUR                                                                                 | RESTART               |
|--------------------------------------------------|----------------------|-------------------------------------------------------------------------------------------|-----------------------|
| <b>Smart</b><br><code>-m smart</code>            | <code>SIGTERM</code> | Refuses new connections, <b>waits for every existing session to disconnect on its own</b> | Clean                 |
| <b>Fast</b><br><code>-m fast</code><br>(default) | <code>SIGINT</code>  | Rolls back active transactions, disconnects clients, checkpoints, exits                   | Clean                 |
| <b>Immediate</b><br><code>-m immediate</code>    | <code>SIGQUIT</code> | Kills every process at once. <b>No checkpoint.</b>                                        | <b>Crash recovery</b> |

```
pg_ctl -D $PGDATA stop -m fast -t 120 -w      # -w wait, -t timeout
pg_ctl -D $PGDATA restart -m fast -w
pg_ctl -D $PGDATA reload                      # SIGHUP – re-read config, no downtime
pg_ctl -D $PGDATA status
pg_ctl -D $PGDATA promote                    # end recovery on a standby

systemctl stop postgresql-17                 # uses fast by default in PGDG units
```

### Choosing

- **Fast** is the correct default. Clients see their connection closed; uncommitted work is rolled back; the next start needs no recovery because a clean shutdown checkpoint was written.
- **Smart** waits indefinitely for a single idle client. On a system with a connection pool it may never complete. Use it only when you genuinely must let sessions finish.
- **Immediate** is a simulated crash. The next start replays WAL from the last checkpoint. It is safe — WAL guarantees consistency — but startup takes longer and it is the wrong choice if a clean option is available.

### What recovery looks like after an immediate stop

```
LOG:  database system was interrupted; last known up at 2026-09-14 10:22:11
LOG:  database system was not properly shut down; automatic recovery in progress
LOG:  redo starts at 3/17000028
LOG:  invalid record length at 3/1706A120: wanted 24, got 0
LOG:  redo done at 3/1706A0F8, elapsed time: 4.21 s
LOG:  database system is ready to accept connections
```

Recovery duration is governed by how much WAL accumulated since the last checkpoint — so a large `max_wal_size` trades faster steady-state operation against slower crash recovery. That is a deliberate choice, not an oversight.

### Shutdown that hangs

```
# Who is still connected?
psql -c "SELECT pid, username, state, now()-state_change AS idle_for, left(query,60)
        FROM pg_stat_activity WHERE backend_type='client backend';"

# Ask them to leave
psql -c "SELECT pg_terminate_backend(pid) FROM pg_stat_activity
        WHERE backend_type='client backend' AND pid <> pg_backend_pid();"

# A long checkpoint can also delay a fast shutdown – check the log
tail -f $PGDATA/log/postgresql-*.log | grep -i checkpoint
```

**NEVER SIGKILL THE POSTMASTER**

`kill -9` on the postmaster leaves orphaned backend processes still attached to shared memory and a stale `postmaster.pid`. The next start may refuse, or worse, a second postmaster may attach to the same data directory. If a shutdown is truly stuck, use `-m immediate` — that is exactly what it is for.

## 19 User Administration 1 — Roles, users, privileges

### CORE

PostgreSQL has only roles. “User” means a role with `LOGIN`; “group” means one without. There is no separate object type, and `CREATE USER` is literally an alias for `CREATE ROLE ... LOGIN`.

### Role attributes

```
CREATE ROLE xxxx_app WITH
    LOGIN
    PASSWORD 'strong-password'
    CONNECTION LIMIT 50
    VALID UNTIL '2027-01-01'
    NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT NOREPLICATION NOBYPASSRLS;

ALTER ROLE xxxx_app NOLOGIN;           -- disable without dropping
ALTER ROLE xxxx_app PASSWORD NULL;     -- remove the password
ALTER ROLE xxxx_app SET statement_timeout = '5min'; -- per-role default
\du+
```

| ATTRIBUTE                          | GRANTS                                                                             |
|------------------------------------|------------------------------------------------------------------------------------|
| <code>SUPERUSER</code>             | Bypasses every permission check. Grant to almost nobody.                           |
| <code>CREATEDB / CREATEROLE</code> | Create databases / create and alter roles                                          |
| <code>REPLICATION</code>           | Open a replication connection — needed for standbys and <code>pg_basebackup</code> |
| <code>BYPASSRLS</code>             | Ignore row-level security policies                                                 |
| <code>INHERIT</code>               | Automatically use privileges of granted roles (default)                            |

### Privileges

```
-- Database and schema
GRANT CONNECT, TEMPORARY ON DATABASE xxxx TO xxxx_app;
GRANT USAGE, CREATE ON SCHEMA app TO xxxx_owner;

-- Tables, sequences, functions
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO xxxx_rw;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA app TO xxxx_rw;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA app TO xxxx_rw;

-- Column level
GRANT SELECT (order_id, status) ON xxxx_orders TO xxxx_analyst;
GRANT UPDATE (status) ON xxxx_orders TO xxxx_app;

-- FUTURE objects – the step that is always forgotten
ALTER DEFAULT PRIVILEGES FOR ROLE xxxx_owner IN SCHEMA app
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO xxxx_rw;
ALTER DEFAULT PRIVILEGES FOR ROLE xxxx_owner IN SCHEMA app
    GRANT USAGE, SELECT ON SEQUENCES TO xxxx_rw;
```

#### ALTER DEFAULT PRIVILEGES IS SCOPED TO A GRANTOR

It only affects objects created **by the role that ran it**. If your deployment creates tables as `xxxx_owner` but you ran the statement as `postgres`, every new table will be invisible to the application role and somebody will re-run grants after each release. Always use `FOR ROLE`.

## Inspecting effective access

```
\dp app.xxxx_orders
SELECT has_table_privilege('xxx_app','app.xxxx_orders','SELECT');
SELECT * FROM information_schema.role_table_grants WHERE grantee = 'xxx_rw';

-- Role membership graph
SELECT r.rolname AS role, m.rolname AS member_of
FROM   pg_auth_members am
JOIN   pg_roles r ON r.oid = am.member
JOIN   pg_roles m ON m.oid = am.roleid
ORDER BY 1;
```

## Dropping a role that owns objects

```
-- A role cannot be dropped while it owns anything or holds any privilege
REASSIGN OWNED BY xxx_old TO xxx_owner;  -- in EVERY database
DROP OWNED BY xxx_old;                   -- removes remaining privileges
DROP ROLE xxx_old;
```



## 20 User Administration 2 — RBAC and Row-Level Security

CORE

### A role hierarchy that scales



```

-- Groups
CREATE ROLE xxxx_owner NOLOGIN;
CREATE ROLE xxxx_readonly NOLOGIN;
CREATE ROLE xxxx_readwrite NOLOGIN;
CREATE ROLE xxxx_admin NOLOGIN;

-- Privileges attach to GROUPS only
GRANT CONNECT ON DATABASE xxxx TO xxxx_readonly, xxxx_readwrite, xxxx_admin;
GRANT USAGE ON SCHEMA app TO xxxx_readonly, xxxx_readwrite, xxxx_admin;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO xxxx_readonly;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO xxxx_readwrite;
GRANT ALL ON ALL TABLES IN SCHEMA app TO xxxx_admin;
ALTER DEFAULT PRIVILEGES FOR ROLE xxxx_owner IN SCHEMA app
    GRANT SELECT ON TABLES TO xxxx_readonly;

-- Logins are only ever members
CREATE ROLE xxxx_app LOGIN PASSWORD '...' IN ROLE xxxx_readwrite;
CREATE ROLE xxxx_report LOGIN PASSWORD '...' IN ROLE xxxx_readonly;

-- Joiner / leaver becomes one statement
GRANT xxxx_admin TO xxxx_dba_majid;
REVOKE xxxx_admin FROM xxxx_dba_majid;

```

### Row-Level Security

RLS filters **rows** by policy, enforced by the server rather than by the application. Every query against the table — from any client, any tool — is filtered.

```

ALTER TABLE app.xxxx_orders ENABLE ROW LEVEL SECURITY;
-- Optional: apply even to the table owner
ALTER TABLE app.xxxx_orders FORCE ROW LEVEL SECURITY;

-- READ policy
CREATE POLICY branch_select ON app.xxxx_orders
  FOR SELECT TO xxxx_readwrite
  USING (branch_id = current_setting('app.branch_id', true)::int);

-- WRITE policy: USING controls which rows may be changed,
--               WITH CHECK controls what the resulting row may look like
CREATE POLICY branch_modify ON app.xxxx_orders
  FOR ALL TO xxxx_readwrite
  USING      (branch_id = current_setting('app.branch_id', true)::int)
  WITH CHECK (branch_id = current_setting('app.branch_id', true)::int);

-- The application sets the context per transaction
SET LOCAL app.branch_id = '42';
SELECT count(*) FROM app.xxxx_orders;  -- only branch 42 is visible

\d app.xxxx_orders
SELECT * FROM pg_policies WHERE tablename = 'xxx_orders';

```

| CLAUSE                  | APPLIES TO                                                       |
|-------------------------|------------------------------------------------------------------|
| USING                   | Which existing rows are visible or modifiable                    |
| WITH CHECK              | Which new or updated rows are permitted                          |
| PERMISSIVE<br>(default) | Policies combine with <b>OR</b>                                  |
| RESTRICTIVE             | Policies combine with <b>AND</b> — use to add a mandatory filter |

### THREE RLS FACTS TO KNOW BEFORE DEPLOYING IT

1. Table owners bypass RLS unless you add **FORCE ROW LEVEL SECURITY**. 2. Superusers and roles with **BYPASSRLS** always bypass it. 3. Policy expressions are evaluated per row and can defeat index usage — check **EXPLAIN** on your hottest queries after enabling it, and make sure the policy column is indexed.

## Auditing

```

-- Who is a superuser? Should be a very short list.
SELECT rolname FROM pg_roles WHERE rolsuper;

-- Roles with no expiry and no connection limit
SELECT rolname, rolvaliduntil, rolconlimit FROM pg_roles
WHERE rolcanlogin AND rolvaliduntil IS NULL;

-- Full statement auditing, when required for compliance
-- shared_preload_libraries = 'pgaudit'
CREATE EXTENSION pgaudit;
ALTER SYSTEM SET pgaudit.log = 'write, ddl, role';
ALTER ROLE xxxx_dba_majid SET pgaudit.log = 'all';

```

## PHASE 2

# Backup, Replication and High Availability

Sessions 21–29

---

The most critical production skills. The only area where being wrong is unrecoverable.

**21** pg\_dump — Logical backup fundamentals

---

**22** pg\_basebackup 1 (Theory) — Physical backup fundamentals

---

**23** pg\_basebackup 2 (Scenarios) — Hands-on backup and restore

---

**24** Replication Slot Based Replication

---

**25** pg\_basebackup 3 (Replication) — Standby provisioning

---

**26** Upgradation 1 — pg\_upgrade and major version upgrades

---

**27** Upgradation 2 — Logical replication based upgrades

---

**28** PgPool-II and PgBouncer — Pooling and load balancing

---

**29** Patroni — Automatic failover and HA cluster setup

---

## 21 pg\_dump — Logical backup fundamentals

### CORE

**What it is.** A consistent, single-transaction logical export of one database. It runs against a live server without blocking writers, because it takes an MVCC snapshot.

### Formats — and why plain SQL is almost never the right choice

| FLAG | FORMAT        | CHARACTERISTICS                                                                                         |
|------|---------------|---------------------------------------------------------------------------------------------------------|
| -Fp  | Plain SQL     | Human-readable. No selective restore, no parallel restore, no compression. Default, and rarely correct. |
| -Fc  | <b>Custom</b> | Compressed, single file, <b>selective and parallel restore. The default you should use.</b>             |
| -Fd  | Directory     | One file per table. The <b>only format supporting parallel dump</b> ( -j ).                             |
| -Ft  | Tar           | Selective restore, no compression. Rarely useful.                                                       |

```
# The everyday command
pg_dump -h HOST -U postgres -d xxxx -Fc -Z6 -v -f xxxx_$(date +%F).dump

# Parallel dump — directory format only
pg_dump -h HOST -U postgres -d xxxx -Fd -j 8 -f /backup/xxxx_dir

# Globals are NOT in pg_dump. Roles and tablespaces live at cluster level.
pg_dumpall -h HOST -U postgres --globals-only -f globals.sql
```

### A PG\_DUMP ALONE IS NOT A RESTORABLE BACKUP

It contains no roles, no tablespaces, no other databases. Restore it into a clean cluster and every object comes back owned by whoever ran the restore, and every grant fails. **Always pair it with `pg_dumpall --globals-only`**, and restore the globals first.

### Selective options worth knowing

```
-n app          --schema=app          only this schema
-N audit        --exclude-schema=audit
-t xxxx_orders  --table=xxxx_orders    only this table
-T 'xxxx_log*'  --exclude-table        glob patterns accepted
--schema-only / --data-only
--exclude-table-data='xxxx_audit*'     structure but not the rows
--no-owner --no-privileges             for restoring into a different estate
--no-comments
--section=pre-data | data | post-data  split DDL, rows, and index/constraint DDL
--snapshot=NAME                       share a snapshot across parallel jobs
```

## Restoring

```
pg_restore -l xxxx.dump | head -40          # inspect first, always
pg_restore -h HOST -U postgres -d xxxx_new -j 8 -v xxxx.dump

# One table out of a full dump
pg_restore -h HOST -U postgres -d xxxx -t xxxx_orders --data-only xxxx.dump

# Selective restore driven by an edited list
pg_restore -l xxxx.dump > toc.txt
#   edit toc.txt, prefix unwanted lines with ';'
pg_restore -L toc.txt -d xxxx_new xxxx.dump

# Plain-format dumps go through psql, not pg_restore
psql -v ON_ERROR_STOP=1 -d xxxx_new -f xxxx.sql

# MANDATORY afterwards — pg_restore does not build statistics
psql -d xxxx_new -c "ANALYZE;"
```

## Where pg\_dump fits, and where it does not

| USE IT FOR                                                 | DO NOT USE IT FOR                                                  |
|------------------------------------------------------------|--------------------------------------------------------------------|
| Single-table or single-schema recovery                     | Disaster recovery of a large cluster — restore time is the blocker |
| Migrating between major versions or platforms              | Point-in-time recovery — it has no concept of a target time        |
| Refreshing a lower environment                             | Building a streaming standby                                       |
| A version-portable escape hatch alongside physical backups | Anything where RTO is measured in minutes                          |

### THE CONSISTENCY GUARANTEE, PRECISELY

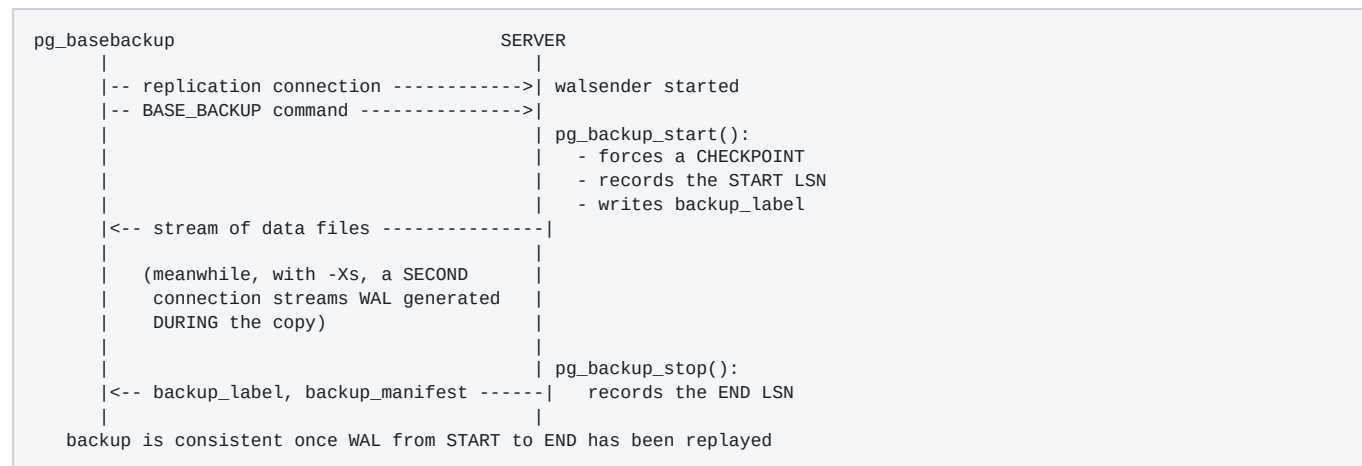
`pg_dump` is consistent as of the moment it **started**, because it holds one repeatable-read snapshot for its whole run. It does not block writers — but it does hold back the xmin horizon for its entire duration, so a twelve-hour dump prevents `VACUUM` from reclaiming anything for twelve hours. On large databases that is a real, and frequently overlooked, cost.

## 22 pg\_basebackup 1 (Theory) — Physical backup fundamentals

### CORE

**What it is.** A byte-level copy of the entire data directory, taken over a replication connection, together with enough WAL to make it consistent. It is the foundation for both PITR and standby provisioning.

### How it works internally



The copy is not an instantaneous snapshot — files change while they are being read. Consistency comes from replaying the WAL generated during the copy. This is why **the WAL accompanying a base backup is not optional**; without it the directory is an unusable set of torn files.

### The options that matter

| OPTION                               | MEANING                                                                                                    |
|--------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>-D dir</code>                  | Destination. Must be empty.                                                                                |
| <code>-Fp / -Ft</code>               | Plain directory / tar. <b>Plain is required for PG 17 incremental backups.</b>                             |
| <code>-X stream (-Xs)</code>         | <b>Stream WAL on a second connection.</b> The default and the right choice — the backup is self-contained. |
| <code>-X fetch</code>                | Collect WAL at the end. Fails if a segment was recycled first.                                             |
| <code>-c fast</code>                 | Immediate checkpoint. Without it the backup waits for a spread checkpoint.                                 |
| <code>-R</code>                      | Write <code>standby.signal</code> and <code>primary_conninfo</code> — produces a ready-to-start standby.   |
| <code>-S slot / -C</code>            | Use a replication slot / create it if absent.                                                              |
| <code>-z / -Z level</code>           | Compression (tar format; PG 15+ also supports server-side and LZ4/zstd).                                   |
| <code>-P -v</code>                   | Progress and verbose output.                                                                               |
| <code>-T old=new</code>              | Relocate a tablespace.                                                                                     |
| <code>-- incremental=MANIFEST</code> | <b>PostgreSQL 17.</b> Block-level incremental against a prior backup.                                      |

## Prerequisites on the server

```
wal_level = replica          -- or logical
max_wal_senders = 10         -- -Xs needs TWO: one for data, one for WAL
max_replication_slots = 10

CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'strong-password';
-- pg_hba.conf:
host      replication      replicator      10.20.30.0/24      scram-sha-256
```

## What lands in the backup directory

```
backup_label      START LSN, checkpoint location, backup method and start time
backup_manifest   every file with its size and checksum – the input to pg_verifybackup
base/ global/ pg_wal/ pg_xact/ ...   the data directory itself
postgresql.conf, pg_hba.conf         the SOURCE server's configuration – review it before starting
```

### PG\_DUMP VERSUS PG\_BASEBACKUP, IN ONE LINE EACH

`pg_dump` answers “*give me this table back*”. `pg_basebackup` answers “*give me the whole server back, as it was at 14:02*”. A production estate needs both, and they are not substitutes.

## 23 pg\_basebackup 2 (Scenarios) — Hands-on backup and restore

### CORE

#### Scenario A — a compressed full backup for archive

```
pg_basebackup -h HOST -U replicator \
  -D /backup/base/full_$(date +%Y%m%d) \
  -Ft -z -Z6 -Xs -c fast -P -v

ls -la /backup/base/full_20260914/
#  base.tar.gz  pg_wal.tar.gz  backup_manifest
```

#### Scenario B — plain format, verified

```
pg_basebackup -h HOST -U replicator -D /backup/base/full_20260914 -Xs -c fast -P -v

pg_verifybackup /backup/base/full_20260914
#  expect exactly:  backup successfully verified
```

#### Scenario C — PostgreSQL 17 incremental chain

```
-- Prerequisite, BEFORE the full backup is taken
ALTER SYSTEM SET summarize_wal = on;
ALTER SYSTEM SET wal_summary_keep_time = '10d';
SELECT pg_reload_conf();
SHOW summarize_wal;                -- must be 'on'

-- Confirm summaries cover the intended base
SELECT tli, start_lsn, end_lsn FROM pg_available_wal_summaries() ORDER BY start_lsn LIMIT 5;
```

```
# Full, then incrementals chained off the PREVIOUS backup's manifest
pg_basebackup -D /backup/base/full_20260914 -c fast -P -v

pg_basebackup -D /backup/base/incr_20260915 \
  --incremental=/backup/base/full_20260914/backup_manifest -c fast -P -v

pg_basebackup -D /backup/base/incr_20260916 \
  --incremental=/backup/base/incr_20260915/backup_manifest -c fast -P -v

# Prove it really is incremental
find /backup/base/incr_20260915/base -name 'INCREMENTAL.*' | wc -l

# Restore: MERGE the chain, in order
pg_combinebackup /backup/base/full_20260914 \
  /backup/base/incr_20260915 \
  /backup/base/incr_20260916 \
  -o /var/lib/pgsql/17/data
```

#### TWO RULES FOR INCREMENTAL CHAINS

1. `summarize_wal` must have been on **when the parent backup was taken** — enabling it afterwards does not retrofit.
2. Delete any link and every backup after it becomes unrestoreable. Start a fresh full weekly so the chain stays short.



## Scenario D — restoring a full backup

```
systemctl stop postgresql-17
mv /var/lib/postgresql/17/data /var/lib/postgresql/17/data.old
mkdir -p /var/lib/postgresql/17/data && chmod 700 /var/lib/postgresql/17/data

tar -xzf /backup/base/full_20260914/base.tar.gz -C /var/lib/postgresql/17/data
tar -xzf /backup/base/full_20260914/pg_wal.tar.gz -C /var/lib/postgresql/17/data/pg_wal

chown -R postgres:postgres /var/lib/postgresql/17/data
# REVIEW the restored postgresql.conf and pg_hba.conf – they came from the SOURCE server
systemctl start postgresql-17
tail -f /var/lib/postgresql/17/data/log/postgresql-*.log
```

## Scenario E — point-in-time recovery

```
# Restore the base backup taken BEFORE the target time, then:
cat >> $PGDATA/postgresql.conf <<'EOF'
restore_command      = 'cp /wal_archive/%f %p'
recovery_target_time  = '2026-09-14 13:59:59+05:30'
recovery_target_action = 'pause'
EOF
touch $PGDATA/recovery.signal
systemctl start postgresql-17

psql -c "SELECT pg_is_in_recovery();"          -- true, paused at the target
psql -d xxxx -c "SELECT count(*) FROM xxxx_orders;" -- VERIFY before promoting
psql -c "SELECT pg_promote();"                 -- ends recovery, opens read-write
```

Other targets: `recovery_target_xid`, `recovery_target_lsn`, `recovery_target_name` (created earlier by `pg_create_restore_point('pre_release')`), and `recovery_target = 'immediate'`.

## Common failures

| ERROR                                                                        | CAUSE                                                                                  |
|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| directory "... " exists but is not empty                                     | <code>-D</code> must point at an empty directory                                       |
| could not connect: FATAL: no pg_hba.conf entry ... replication               | Missing the replication <code>pg_hba</code> line                                       |
| number of requested standby connections exceeds <code>max_wal_senders</code> | <code>-Xs</code> needs two senders; raise the setting                                  |
| requested WAL segment has already been removed                               | Used <code>-X fetch</code> with insufficient retention; use <code>-Xs</code> or a slot |
| WAL summaries are not available for the requested range                      | <code>summarize_wal</code> was off when the parent backup ran                          |

## 24 Replication Slot Based Replication

### CORE

A **replication slot** is a **server-side promise**: the primary will not discard WAL, and will not vacuum away rows, that a named consumer has not yet confirmed. It converts “I hope the standby keeps up” into a guarantee.

| TYPE     | CONSUMER                                    | ALSO HOLDS BACK                                                       |
|----------|---------------------------------------------|-----------------------------------------------------------------------|
| Physical | A streaming standby                         | WAL only                                                              |
| Logical  | Logical replication, Debezium and other CDC | WAL <b>and the catalog xmin</b> — so it can block VACUUM cluster-wide |

### Creating and using one

```
-- On the primary
SELECT pg_create_physical_replication_slot('standby1_slot');
SELECT * FROM pg_replication_slots;

-- Provision the standby, creating and using the slot in one step
pg_basebackup -h primary.host -U replicator -D $PGDATA \
    -Xs -P -v -R -C -S standby1_slot

-- -R writes into postgresql.auto.conf:
--     primary_conninfo = 'host=primary.host user=replicator ...'
--     primary_slot_name = 'standby1_slot'
--     and creates standby.signal
```

### With a slot versus without

| STANDBY OFFLINE FOR HOURS                  | RESULT                                                                                                                         |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Slot in use                                | Primary retains every needed segment. Standby reconnects and catches up. <b>Risk:</b> <code>pg_wal</code> grows without bound. |
| <code>wal_keep_size</code> only            | Retains a fixed amount. Beyond it: requested WAL segment has already been removed.                                             |
| WAL archive + <code>restore_command</code> | Standby fetches the gap from the archive. <b>The resilient design.</b>                                                         |
| None of the above                          | Rebuild the standby from a fresh base backup.                                                                                  |

### The guard that makes slots safe

```
-- PostgreSQL 13+: cap how much a slot may retain
ALTER SYSTEM SET max_slot_wal_keep_size = '50GB';
SELECT pg_reload_conf();

SELECT slot_name, slot_type, active, active_pid, wal_status, safe_wal_size,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained,
       age(xmin) AS xmin_age, age(catalog_xmin) AS catalog_xmin_age
FROM   pg_replication_slots
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
-- wal_status: reserved | extended | unreserved | lost
```

Once the cap is exceeded the slot becomes `lost` and its consumer can no longer catch up from WAL. That is a deliberate trade: **sacrifice one standby rather than the primary's disk**. Pair it with WAL archiving so the standby can still recover from the archive.

**THE NUMBER ONE CAUSE OF “THE DISK FILLED OVERNIGHT”**

An inactive slot left behind by a decommissioned standby or a switched-off CDC consumer. The primary retains WAL forever; `pg_wal` grows until the filesystem is full and the database PANICS. A logical slot additionally freezes the xmin horizon, so the whole cluster bloats at the same time. **Alert on `active = false`.**

```
-- Remove a slot whose consumer is genuinely gone
SELECT pg_drop_replication_slot('old_standby_slot');
CHECKPOINT;

-- Logical replication uses slots too
CREATE PUBLICATION xxxx_pub FOR TABLE app.xxxx_orders, app.xxxx_customers;
CREATE SUBSCRIPTION xxxx_sub
    CONNECTION 'host=primary.host dbname=xxxx user=replicator password=...'
    PUBLICATION xxxx_pub;      -- creates a logical slot on the publisher automatically
SELECT * FROM pg_stat_subscription;
```

## 25 pg\_basebackup 3 (Replication) — Standby provisioning

CORE

### End-to-end build of a streaming standby

```
# ----- ON THE PRIMARY -----
# postgresql.conf
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = '2GB'
hot_standby = on
archive_mode = on
archive_command = 'test ! -f /wal_archive/%f && cp %p /wal_archive/%f'
wal_log_hints = on          # REQUIRED for pg_rewind later – set it now
synchronous_commit = on
# requires a RESTART for wal_level and max_wal_senders
```

```
-- On the primary
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'strong-password';
SELECT pg_create_physical_replication_slot('standby1_slot');
```

```
# pg_hba.conf on the primary
host      replication    replicator    10.20.30.41/32    scram-sha-256
```

```
# ----- ON THE STANDBY -----
systemctl stop postgresql-17
rm -rf /var/lib/pgsql/17/data/*

pg_basebackup -h primary.host -U replicator \
  -D /var/lib/pgsql/17/data \
  -Xs -P -v -R -C -S standby1_slot

# Verify what -R generated
cat /var/lib/pgsql/17/data/postgresql.auto.conf
ls /var/lib/pgsql/17/data/standby.signal

# Add the archive fallback – this is what saves you after a long outage
cat >> /var/lib/pgsql/17/data/postgresql.auto.conf <<'EOF'
restore_command = 'cp /wal_archive/%f %p'
EOF

chown -R postgres:postgres /var/lib/pgsql/17/data
systemctl start postgresql-17
```

### Verification from both sides

```
-- PRIMARY
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       write_lag, flush_lag, replay_lag,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS behind
FROM   pg_stat_replication;

-- STANDBY
SELECT pg_is_in_recovery();           -- true
SELECT pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn(),
       pg_last_xact_replay_timestamp();
SELECT status, sender_host, slot_name, latest_end_lsn FROM pg_stat_wal_receiver;
```

## Standby-specific settings

```
hot_standby = on                -- allow read-only queries
max_standby_streaming_delay = '30s' -- how long replay waits for a conflicting query
max_standby_archive_delay = '30s'
hot_standby_feedback = off      -- see the trade-off below
wal_receiver_status_interval = '10s'
recovery_min_apply_delay = '0'  -- set to e.g. '1h' for a delayed standby
```

### THE TRADE-OFF YOU MUST CHOOSE CONSCIOUSLY

A long read query on the standby conflicts with replay. With `hot_standby_feedback = off`, the query is **cancelled** after `max_standby_streaming_delay` — the standby stays current and the reporting user loses their query. With it `on`, the standby tells the primary to hold back its xmin horizon — the query survives and **the primary bloats**. There is no setting that avoids the cost; decide which side pays and document it.

## Synchronous replication

```
-- On the primary
ALTER SYSTEM SET synchronous_standby_names = 'ANY 1 (standby1, standby2)';
ALTER SYSTEM SET synchronous_commit = 'on';
SELECT pg_reload_conf();
SELECT application_name, sync_state FROM pg_stat_replication;
```

### A SINGLE SYNCHRONOUS STANDBY IS A LIABILITY

If it goes down, **every commit on the primary hangs** — you chose zero data loss over availability. Run at least two sync candidates with `ANY 1 (...)`, or have a rehearsed procedure to clear `synchronous_standby_names` during an outage.

## A delayed standby — cheap insurance against human error

```
-- On a third standby
ALTER SYSTEM SET recovery_min_apply_delay = '1h';
SELECT pg_reload_conf();
```

It stays one hour behind. If someone truncates a table at 14:00, you have until 15:00 to pause replay and extract the data — far faster than a full PITR restore.

## 26 Upgradation 1 — pg\_upgrade and major version upgrades

CORE

|                | MINOR (17.2 → 17.4)    | MAJOR (16.X → 17.X)                              |
|----------------|------------------------|--------------------------------------------------|
| What changes   | Bug and security fixes | Features; <b>on-disk catalog format</b>          |
| Data directory | Compatible, untouched  | <b>Must be converted</b>                         |
| Procedure      | Swap binaries, restart | <code>pg_upgrade</code> / dump-restore / logical |
| Downtime       | Seconds                | Minutes to hours                                 |
| Extensions     | Unaffected             | <b>Must be rebuilt for the new major</b>         |

```
# Minor upgrade – this really is all of it
systemctl stop postgresql-17
dnf update -y postgresql17-server postgresql17 postgresql17-contrib
systemctl start postgresql-17
psql -c "SELECT version();"

```

### pg\_upgrade, step by step

```
# 1. Install the new major alongside the old
dnf install -y postgresql17-server postgresql17-contrib postgresql17-devel
/usr/pgsql-17/bin/initdb -D /var/lib/pgsql/17/data --data-checksums \
  --auth-host=scram-sha-256

# 2. CHECK – changes nothing, reports every incompatibility
/usr/pgsql-17/bin/pg_upgrade \
  --old-datadir=/var/lib/pgsql/16/data --new-datadir=/var/lib/pgsql/17/data \
  --old-bindir=/usr/pgsql-16/bin --new-bindir=/usr/pgsql-17/bin \
  --check

# 3. Stop BOTH clusters
systemctl stop postgresql-16 postgresql-17

# 4. Upgrade
/usr/pgsql-17/bin/pg_upgrade \
  --old-datadir=/var/lib/pgsql/16/data --new-datadir=/var/lib/pgsql/17/data \
  --old-bindir=/usr/pgsql-16/bin --new-bindir=/usr/pgsql-17/bin \
  --link --jobs=8
# --link : hard links. Minutes even on multi-TB – but the OLD CLUSTER
#          BECOMES UNUSABLE once the new one starts.
# omit it : copies instead. Slower, needs 2x disk, but the old cluster
#          remains a working rollback path.

# 5. Carry over the configuration you actually changed (pg_upgrade does not)
diff /var/lib/pgsql/16/data/postgresql.conf /var/lib/pgsql/17/data/postgresql.conf
cp /var/lib/pgsql/16/data/pg_hba.conf /var/lib/pgsql/17/data/pg_hba.conf

# 6. Start, then REBUILD STATISTICS – this is mandatory
systemctl start postgresql-17
/usr/pgsql-17/bin/vacuumdb --all --analyze-in-stages --jobs=8

# 7. Only after verification
./delete_old_cluster.sh

```

**THE STEP PEOPLE SKIP, THEN BLAME THE NEW VERSION FOR**

`pg_upgrade` does **not** carry planner statistics across. Without `vacuumdb --analyze-in-stages` the new cluster starts with no statistics at all and every plan is guesswork. `--analyze-in-stages` does three increasingly detailed passes so usable statistics exist within minutes rather than hours.

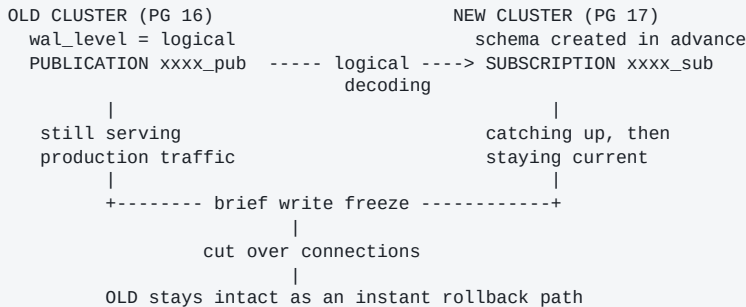
**Pre-upgrade checklist**

| # | CHECK                                     | HOW                                                                           |
|---|-------------------------------------------|-------------------------------------------------------------------------------|
| 1 | Verified, restorable backup               | Take it <b>and test the restore</b>                                           |
| 2 | Release notes for every intervening major | Incompatibilities are listed nowhere else                                     |
| 3 | <code>pg_upgrade --check clean</code>     | Reports without changing anything                                             |
| 4 | Extension inventory                       | <code>SELECT * FROM pg_extension;</code> then confirm each is packaged for 17 |
| 5 | Rehearse on a copy of production, timed   | Not a small dev database                                                      |
| 6 | Disk space                                | Copy mode needs 2×                                                            |
| 7 | Capture plans for critical queries        | To compare afterwards                                                         |
| 8 | Standby strategy                          | Rebuild them, or use <code>rsync --hard-links</code> per the docs             |
| 9 | Written rollback plan with a deadline     | Rehearsed, not theoretical                                                    |

## 27 Upgradation 2 — Logical replication based upgrades

### CORE

**When downtime must be seconds, not minutes** — or when you are changing hardware, operating system or cloud at the same time as the version.



### Procedure

```
-- ===== OLD CLUSTER (publisher) =====
ALTER SYSTEM SET wal_level = 'logical';          -- RESTART required
ALTER SYSTEM SET max_replication_slots = 10;
ALTER SYSTEM SET max_wal_senders = 10;
-- restart, then:
CREATE PUBLICATION xxxx_pub FOR ALL TABLES;
SELECT * FROM pg_publication;
```

```
# ===== NEW CLUSTER: create the schema first =====
pg_dump -h old.host -d xxxx --schema-only -f schema.sql
psql -h new.host -d xxxx -v ON_ERROR_STOP=1 -f schema.sql
# roles too
pg_dumpall -h old.host --globals-only -f globals.sql
psql -h new.host -f globals.sql
```

```
-- ===== NEW CLUSTER (subscriber) =====
CREATE SUBSCRIPTION xxxx_sub
  CONNECTION 'host=old.host dbname=xxxx user=replicator password=...'
  PUBLICATION xxxx_pub
  WITH (copy_data = true, streaming = on);

-- Watch until caught up
SELECT * FROM pg_stat_subscription;
SELECT srrelid::regclass, srsubstate FROM pg_subscription_rel WHERE srsubstate <> 'r';
-- i = initialize, d = data copy, s = synchronised, r = ready
```

```
-- ===== CUTOVER =====
-- 1. Stop application writes
-- 2. Confirm zero lag
SELECT slot_name, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn))
FROM   pg_replication_slots;          -- on the OLD cluster
-- 3. RESET EVERY SEQUENCE – logical replication does NOT replicate them
SELECT format('SELECT setval(%L, (SELECT COALESCE(max(%I),1) FROM %I.%I));',
              s.schemaname||'.'||s.sequencename, a.attname, t.schemaname, t.tablename)
FROM   pg_sequences s, ... ;          -- generate and run on the NEW cluster
-- 4. Repoint the application
-- 5. Drop the subscription
DROP SUBSCRIPTION xxxx_sub;
```



## The limitations that decide whether this is viable

| LIMITATION                       | CONSEQUENCE                                                                      |
|----------------------------------|----------------------------------------------------------------------------------|
| Sequences are not replicated     | Must be reset at cutover, or the first insert fails on a duplicate key           |
| DDL is not replicated            | Freeze schema changes for the whole migration window                             |
| Tables need a replica identity   | A primary key, or <code>ALTER TABLE ... REPLICA IDENTITY FULL</code> (expensive) |
| Large objects are not replicated | Migrate <code>pg_largeobject</code> separately                                   |
| TRUNCATE support                 | PG 11+ only                                                                      |
| Initial copy cost                | A full table copy over the wire; plan the window                                 |

| PG_UPGRADE                 |                                    | LOGICAL REPLICATION                       |
|----------------------------|------------------------------------|-------------------------------------------|
| Downtime                   | Minutes                            | Seconds                                   |
| Complexity                 | Low                                | High                                      |
| Rollback                   | Hard after <a href="#">-- link</a> | Trivial — old cluster is live and current |
| Extra hardware             | None                               | A second server                           |
| Changes OS or platform too | No                                 | Yes                                       |
| Sequences                  | Carried over                       | Manual                                    |

## CHOOSING BETWEEN THEM

`pg_upgrade` for most systems — a short maintenance window is usually acceptable and the procedure is far simpler. Logical replication buys exactly two things: near-zero downtime, and the ability to change platform at the same time. If you need neither, the extra complexity is not worth carrying.

## 28 PgPool-II and PgBouncer — Pooling and load balancing

### CORE

**Why a pooler is not optional.** PostgreSQL forks one OS process per connection. Each carries 1–3 MB plus cached catalog metadata. Two thousand application connections means two thousand processes competing for CPU and memory, and a lock table sized for all of them. A pooler lets 2,000 clients share 50 server connections.

|                        | PGBOUNCER                         | PGPOOL-II                                             |
|------------------------|-----------------------------------|-------------------------------------------------------|
| Primary purpose        | <b>Connection pooling</b>         | Pooling <b>plus</b> load balancing, query routing, HA |
| Architecture           | Single process, async, very light | Multi-process, parses SQL                             |
| Memory footprint       | ~2 KB per connection              | Substantially heavier                                 |
| Read/write splitting   | No                                | <b>Yes</b>                                            |
| Automatic failover     | No                                | Yes (watchdog)                                        |
| Latency added          | Minimal                           | Noticeable — it parses every statement                |
| Operational complexity | Low                               | High                                                  |

#### THE COMMON PRODUCTION PATTERN

**PgBouncer for pooling + HAProxy for routing + Patroni for failover.** Each component does one thing well. PgPool-II tries to do all three and is correspondingly harder to reason about when it misbehaves — though it remains a reasonable single-component choice for smaller estates.

### PgBouncer

```
; /etc/pgbouncer/pgbouncer.ini
[databases]
xxxx = host=primary.host port=5432 dbname=xxxx pool_size=40
xxxx_ro = host=standby.host port=5432 dbname=xxxx pool_size=20

[pgbouncer]
listen_addr = *
listen_port = 6432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 40
reserve_pool_size = 10
reserve_pool_timeout = 5
server_idle_timeout = 600
query_wait_timeout = 120
ignore_startup_parameters = extra_float_digits
admin_users = pgbouncer_admin
stats_users = pgbouncer_stats
```

| POOL_MODE                | SERVER CONNECTION RELEASED     | COST                                                      |
|--------------------------|--------------------------------|-----------------------------------------------------------|
| <code>session</code>     | When the client disconnects    | Safest, least efficient                                   |
| <code>transaction</code> | <b>At each COMMIT/ROLLBACK</b> | <b>The usual choice.</b> Breaks session state — see below |
| <code>statement</code>   | After every statement          | Most efficient; multi-statement transactions impossible   |

**WHAT TRANSACTION POOLING BREAKS**

Session-scoped features stop working because the client is not guaranteed the same backend twice: `SET` (use `SET LOCAL`), session-level advisory locks, `LISTEN / NOTIFY`, `WITH HOLD` cursors, and server-side prepared statements unless the driver disables them. Verify your driver's settings before switching a live application.

```
-- PgBouncer's own admin console
psql -p 6432 -U pgbouncer_admin pgbouncer
SHOW POOLS; SHOW CLIENTS; SHOW SERVERS; SHOW STATS; SHOW CONFIG;
RELOAD; PAUSE xxxx; RESUME xxxx;
```

**PgPool-II, in brief**

```
# pgpool.conf
backend_hostname0 = 'primary.host'
backend_port0 = 5432
backend_weight0 = 1
backend_flag0 = 'ALLOW_TO_FAILOVER'
backend_hostname1 = 'standby.host'
backend_weight1 = 2                # send more reads here

load_balance_mode = on
master_slave_mode = on
master_slave_sub_mode = 'stream'
sr_check_period = 10
health_check_period = 10
failover_command = '/etc/pgpool-II/failover.sh %d %P %H %R'
```

```
psql -h pgpool.host -p 9999 -c "SHOW POOL_NODES;"
```

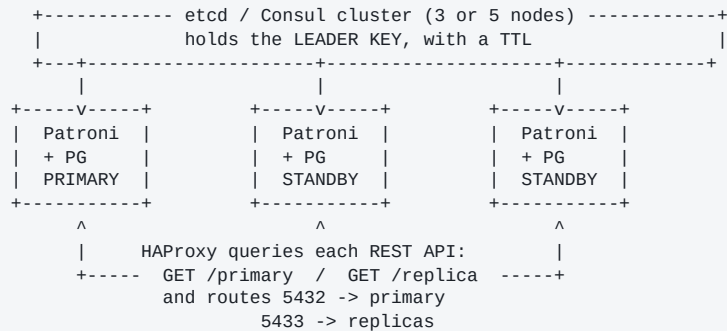
**Sizing the pool**

```
server pool size ≈ (number of CPU cores × 2) + effective_spindle_count
-- For 16 cores on SSD: roughly 35-40 server connections
-- max_client_conn can be 50-100x that; that is the entire point.
```

## 29 Patroni — Automatic failover and HA cluster setup

### CORE

**What Patroni is.** A Python daemon running beside each PostgreSQL instance that decides **who is primary**, and enforces that decision through a distributed consensus store. It does not replace streaming replication — it orchestrates it.



### How failover actually works

1. The primary's Patroni renews the **leader key** in etcd every `loop_wait` seconds. The key carries a TTL.
2. If the primary dies or is partitioned, it stops renewing and **the key expires**.
3. Standby Patronis detect the expiry, compare WAL positions, and the most advanced eligible node **races to acquire the key**.
4. Only one can win — etcd requires a quorum. The winner promotes PostgreSQL.
5. The remaining nodes reconfigure to follow the new leader, running `pg_rewind` if they diverged.
6. HAProxy's health check observes the change and moves write traffic.

**Why this prevents split brain:** a node in a minority partition **cannot obtain the leader key**, and Patroni demotes any PostgreSQL instance that finds itself primary without holding it.

## Configuration

```
# /etc/patroni.yml
scope: xxxx-cluster
name: node-a

restapi:
  listen: 0.0.0.0:8008
  connect_address: 10.20.30.41:8008

etcd3:
  hosts: 10.20.30.11:2379,10.20.30.12:2379,10.20.30.13:2379

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576          # 1 MB – refuse to promote a node further behind
    synchronous_mode: true
    synchronous_mode_strict: false           # true = block writes rather than lose sync
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: replica
        hot_standby: "on"
        max_wal_senders: 10
        max_replication_slots: 10
        wal_log_hints: "on"                  # REQUIRED for pg_rewind
        archive_mode: "on"
        archive_command: pgbackrest --stanza=xxxx archive-push %p
  initdb:
    - encoding: UTF8
    - data-checksums

postgresql:
  listen: 0.0.0.0:5432
  connect_address: 10.20.30.41:5432
  data_dir: /var/lib/pgsql/17/data
  bin_dir: /usr/pgsql-17/bin
  authentication:
    replication: {username: replicator, password: '...'}
    superuser: {username: postgres, password: '...')}
```

## Day-to-day operation

```
patronictl -c /etc/patroni.yml list
patronictl -c /etc/patroni.yml switchover --leader node-a --candidate node-b
patronictl -c /etc/patroni.yml failover --candidate node-b
patronictl -c /etc/patroni.yml reinit node-c          # rebuild a broken member
patronictl -c /etc/patroni.yml pause                  # suspend automation for maintenance
patronictl -c /etc/patroni.yml resume
patronictl -c /etc/patroni.yml edit-config            # change PG parameters cluster-wide
patronictl -c /etc/patroni.yml restart xxxx-cluster node-a
patronictl -c /etc/patroni.yml query -c "SELECT pg_is_in_recovery();"
```

```
# HAProxy: one write endpoint, one read endpoint
listen postgres_write
    bind *:5432
    option httpchk GET /primary
    http-check expect status 200
    default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
    server node-a 10.20.30.41:5432 check port 8008
    server node-b 10.20.30.42:5432 check port 8008
    server node-c 10.20.30.43:5432 check port 8008

listen postgres_read
    bind *:5433
    balance roundrobin
    option httpchk GET /replica
    http-check expect status 200
    server node-b 10.20.30.42:5432 check port 8008
    server node-c 10.20.30.43:5432 check port 8008
```

#### ALWAYS PAUSE BEFORE MAINTENANCE

`patronictl pause` before any manual `pg_ctl` operation, configuration edit or planned restart. Without it, Patroni sees an unexpected state, concludes the node has failed, and initiates a failover in the middle of your maintenance. Remember to `resume` afterwards — a cluster left paused has no automatic failover at all.

PHASE 3

# Performance Tuning

Sessions 30–35

Three sessions of parameter tuning and three of log analysis. Change one thing, measure, keep or revert.

|    |                                                  |
|----|--------------------------------------------------|
| 30 | Parameter Tuning 1 — Memory and performance      |
| 31 | Parameter Tuning 2 — WAL, checkpoint and planner |
| 32 | Parameter Tuning 3 — OS-level and advanced       |

|    |                                                   |
|----|---------------------------------------------------|
| 33 | pgBadger 1 — Setup and basic log analysis         |
| 34 | pgBadger 2 — Advanced query performance reporting |
| 35 | pgBadger 3 — Production case studies              |

## 30 Parameter Tuning 1 — Memory and performance

### CORE

**Principle.** Change one thing, measure, keep or revert. A configuration copied from a blog is a configuration you cannot defend when it misbehaves.

#### The memory parameters

| PARAMETER                         | DEFAULT | STARTING POINT         | NOTES                                                                             |
|-----------------------------------|---------|------------------------|-----------------------------------------------------------------------------------|
| <code>shared_buffers</code>       | 128MB   | <b>25% of RAM</b>      | Restart. Beyond ~40% you duplicate the OS cache.                                  |
| <code>effective_cache_size</code> | 4GB     | <b>50–75% of RAM</b>   | <b>Allocates nothing</b> — a planner hint. Too low pushes it to sequential scans. |
| <code>work_mem</code>             | 4MB     | 16–64MB                | <b>Per sort/hash node, per worker.</b> Multiplies.                                |
| <code>maintenance_work_mem</code> | 64MB    | 1–2GB                  | VACUUM, CREATE INDEX, ALTER TABLE                                                 |
| <code>autovacuum_work_mem</code>  | -1      | 512MB                  | Separate budget per autovacuum worker                                             |
| <code>temp_buffers</code>         | 8MB     | 32MB                   | Per session, for temporary tables                                                 |
| <code>huge_pages</code>           | try     | <code>try or on</code> | Reduces page-table overhead at large <code>shared_buffers</code>                  |
| <code>max_connections</code>      | 100     | 100–300 + pooler       | Restart. Sizes the lock table too.                                                |

#### THE WORK\_MEM FORMULA THAT CAUSES MOST OUT-OF-MEMORY INCIDENTS

$\text{worst case} \approx \text{shared\_buffers} + (\text{work\_mem} \times \text{nodes-per-query} \times \text{parallel\_workers} \times \text{active\_connections})$

A query with three sorts running four parallel workers uses `work_mem` **twelve times over**. At 256 MB with 100 active connections that is 300 GB of theoretical demand. Keep it modest globally; raise it per session for the handful of reports that need it.

```
-- The correct way to give one report more memory
SET work_mem = '512MB';
SELECT ... large aggregation ... ;
RESET work_mem;

-- Or attach it to the reporting role permanently
ALTER ROLE xxxx_report SET work_mem = '256MB';
```

#### Is `work_mem` too small? Ask the database.

```
-- Spills to disk mean work_mem was insufficient
SELECT datname, temp_files, pg_size_pretty(temp_bytes) AS temp_written
FROM   pg_stat_database WHERE temp_files > 0 ORDER BY temp_bytes DESC;

ALTER SYSTEM SET log_temp_files = '10MB';  -- then find them in the log
SELECT pg_reload_conf();

-- And in a plan, look for:
--   Sort Method: external merge  Disk: 248320kB      <- spilled
--   Sort Method: quicksort       Memory: 28kB        <- fitted
```



## Is shared\_buffers right?

```
SELECT round(100.0*sum(blks_hit)/NULLIF(sum(blks_hit)+sum(blks_read),0),2) AS cache_hit_pct
FROM   pg_stat_database;
-- OLTP target: above 95%. Persistently below 90% suggests more shared_buffers,
-- or a working set genuinely larger than RAM.

CREATE EXTENSION pg_buffercache;
SELECT c.relname, count(*) AS buffers,
       pg_size_pretty(count(*) * 8192::bigint) AS size
FROM   pg_buffercache b JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
GROUP BY 1 ORDER BY 2 DESC LIMIT 15;
```

## A worked starting configuration — 32 GB RAM, 8 cores, SSD, OLTP

```
shared_buffers = 8GB           # 25%
effective_cache_size = 24GB    # 75%
work_mem = 32MB
maintenance_work_mem = 2GB
autovacuum_work_mem = 512MB
max_connections = 200          # with PgBouncer in front
huge_pages = try
max_worker_processes = 8
max_parallel_workers = 8
max_parallel_workers_per_gather = 4
max_parallel_maintenance_workers = 4
```

### MEASURE BEFORE AND AFTER, EVERY TIME

`SELECT pg_stat_statements_reset();` before the change, capture the top ten by total time, apply one change, wait for a representative workload, capture again. Without that, you have an opinion rather than a result.

## 31 Parameter Tuning 2 — WAL, checkpoint and planner

CORE

### WAL and checkpoint

| PARAMETER                         | DEFAULT     | PRODUCTION  | EFFECT                                                             |
|-----------------------------------|-------------|-------------|--------------------------------------------------------------------|
| wal_level                         | replica     | replica     | logical only if you need logical replication                       |
| max_wal_size                      | 1GB         | 8–32GB      | Larger = fewer forced checkpoints = less WAL from full-page writes |
| min_wal_size                      | 80MB        | 2–4GB       | Segments recycled rather than deleted                              |
| checkpoint_timeout                | 5min        | 15–30min    | Longer = less total WAL, slower crash recovery                     |
| checkpoint_completion_target      | 0.9         | 0.9         | Spreads writes across the interval                                 |
| wal_compression                   | off         | on          | Compresses full-page images. Large replication-bandwidth saving.   |
| wal_buffers                       | -1          | 16MB        | Auto is usually fine                                               |
| synchronous_commit                | on          | on          | off trades a small loss window for throughput                      |
| full_page_writes                  | on          | on          | <b>Never disable</b> without atomic-write storage                  |
| bgwriter_delay /<br>_lru_maxpages | 200ms / 100 | 100ms / 500 | More background trickle, smaller checkpoint bursts                 |

```
-- Are checkpoints being FORCED rather than scheduled?
SELECT num_timed, num_requested,
       round(100.0*num_requested/NULLIF(num_timed+num_requested,0),1) AS forced_pct,
       write_time, sync_time, buffers_written
FROM   pg_stat_checkpoint;          -- PG16+; earlier pg_stat_bgwriter
```

Above roughly 10% forced means `max_wal_size` is too small. The server is checkpointing because WAL filled, not because time elapsed. Raise it; the cost is disk, the benefit is fewer I/O bursts and less WAL written.

```
-- What the log tells you
-- LOG: checkpoint complete: wrote 24518 buffers (9.3%); ...
--      write=269.6 s, sync=0.1 s, total=270.1 s; ...
-- A short write time with a high buffer count = an I/O spike. Raise the
-- completion target or max_wal_size.
```

### Planner and cost settings

| PARAMETER                 | DEFAULT | SSD             | EFFECT                                                                                                        |
|---------------------------|---------|-----------------|---------------------------------------------------------------------------------------------------------------|
| random_page_cost          | 4.0     | 1.1             | <b>The single highest-value change on SSD.</b> The default assumes spinning disks and suppresses index usage. |
| seq_page_cost             | 1.0     | 1.0             | The baseline everything else is relative to                                                                   |
| cpu_tuple_cost            | 0.01    | 0.01            | Rarely changed                                                                                                |
| effective_io_concurrency  | 1       | 200             | Prefetch depth for bitmap scans                                                                               |
| default_statistics_target | 100     | 100–500         | Histogram detail. Raise per column, not globally.                                                             |
| jit                       | on      | Consider<br>off | Helps long analytical queries; often pure overhead on OLTP                                                    |

| PARAMETER                    | DEFAULT | SSD  | EFFECT                                                                              |
|------------------------------|---------|------|-------------------------------------------------------------------------------------|
| <code>plan_cache_mode</code> | auto    | auto | <code>force_custom_plan</code> fixes generic-plan regressions on partitioned tables |

```
-- Statistics detail on a skewed column, rather than raising the global default
ALTER TABLE xxxx_orders ALTER COLUMN status SET STATISTICS 1000;
ANALYZE xxxx_orders;

-- Extended statistics for correlated columns – the planner assumes independence
CREATE STATISTICS xxxx_orders_corr (dependencies, ndistinct)
  ON branch_id, status FROM xxxx_orders;
ANALYZE xxxx_orders;
```

## Parallelism

```
max_worker_processes = 8           # total pool; restart
max_parallel_workers = 8
max_parallel_workers_per_gather = 4 # per query node
max_parallel_maintenance_workers = 4 # CREATE INDEX, VACUUM
parallel_setup_cost = 1000
parallel_tuple_cost = 0.1
min_parallel_table_scan_size = 8MB
```

### PARALLELISM IS NOT FREE

Each worker gets its own `work_mem`. Raising `max_parallel_workers_per_gather` on a system with a large `work_mem` multiplies memory demand quickly. On OLTP workloads, where queries return few rows, parallelism usually adds setup cost for no gain — consider lowering it rather than raising it.

## 32 Parameter Tuning 3 — OS-level and advanced

### CORE

#### Kernel memory settings

```
# /etc/sysctl.d/30-postgresql.conf
vm.swappiness = 1                # avoid swapping the buffer cache out
vm.overcommit_memory = 2        # refuse overcommit rather than invoke the OOM killer
vm.overcommit_ratio = 80
vm.dirty_background_ratio = 5   # start flushing early, in small amounts
vm.dirty_ratio = 10            # rather than one huge stall
vm.dirty_expire_centisecs = 500
vm.dirty_writeback_centisecs = 100
kernel.shmmax = 68719476736
kernel.shmall = 16777216

sysctl -p /etc/sysctl.d/30-postgresql.conf
```

#### WHY DIRTY\_RATIO MATTERS MORE THAN IT LOOKS

The Linux defaults let a large proportion of RAM accumulate as dirty pages before writeback begins. When the threshold is finally crossed, the kernel flushes everything at once and the entire system stalls — visible as periodic multi-second freezes that correlate with nothing in the database. Lowering both ratios converts one large stall into continuous small writes.

#### Huge pages

```
# Size = shared_buffers / 2MB, plus ~10% headroom
# For shared_buffers = 8GB: 8192/2 = 4096, so allocate ~4500
echo "vm.nr_hugepages = 4500" >> /etc/sysctl.d/30-postgresql.conf
sysctl -p

# postgresql.conf
huge_pages = try                # 'on' refuses to start if unavailable – safer once proven

# Verify after restart
grep Huge /proc/meminfo
SELECT * FROM pg_settings WHERE name = 'huge_pages';
```

#### Transparent huge pages — disable them

```
echo never > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag
# persist via GRUB: transparent_hugepage=never
```

THP causes unpredictable latency spikes as the kernel compacts memory underneath a running database. Explicit huge pages are beneficial; *transparent* huge pages are not.

#### Storage and filesystem

```
# Scheduler: none/noop for NVMe and SSD, mq-deadline for SAS
echo none > /sys/block/nvme0n1/queue/scheduler
echo 256 > /sys/block/nvme0n1/queue/nr_requests
echo 1024 > /sys/block/nvme0n1/queue/read_ahead_kb

# Mount options
/dev/mapper/vg0-pgdata /var/lib/pgsql xfs noatime,nodiratime,logbufs=8 0 0
```

| FILESYSTEM         | VERDICT                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------|
| <b>XFS</b>         | The most common production choice. Good large-file and parallel behaviour.                           |
| <b>ext4</b>        | Equally supported and well tested. Mount with <code>data=ordered</code> .                            |
| <b>ZFS</b>         | Compression and snapshots are attractive; set <code>recordsize=8K</code> and disable double caching. |
| <b>btrfs / NFS</b> | Avoid for PGDATA.                                                                                    |

### Protecting the postmaster from the OOM killer

```
# In the systemd unit, or a drop-in:
[Service]
OOMScoreAdjust=-1000
Environment=PG_OOM_ADJUST_FILE=/proc/self/oom_score_adj
Environment=PG_OOM_ADJUST_VALUE=0
# The postmaster is protected; individual backends remain killable,
# which is exactly what you want.
```

### CPU, NUMA and network

```
cpupower frequency-set -g performance      # no frequency scaling on a DB server
numactl --hardware                          # check the topology
# postgresql.conf if the server is NUMA and shared_buffers spans nodes:
#   consider numactl --interleave=all in the unit file

# Network, for replication over a busy link
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
net.ipv4.tcp_slow_start_after_idle = 0
```

### Verifying the whole picture

```
SELECT name, setting, unit, source, sourcefile, sourceline, pending_restart
FROM   pg_settings WHERE source NOT IN ('default','override') ORDER BY name;

-- pending_restart = t means the value is staged but NOT in force
SELECT name, setting FROM pg_settings WHERE pending_restart;
```

## 33 pgBadger 1 — Setup and basic log analysis

### CORE

**What pgBadger is.** A Perl log analyser that turns PostgreSQL's log into an interactive HTML report: slowest queries, most frequent queries, temporary files, locks, checkpoints, connections, errors — with time-series charts. It reads the log and nothing else, so it adds zero load to the database.

### The log configuration pgBadger requires

```
logging_collector = on
log_destination = 'stderr'
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_rotation_age = '1d'
log_rotation_size = 0

# THIS PREFIX IS MANDATORY — pgBadger cannot parse the log without it
log_line_prefix = '%t [%p]: user=%u,db=%d,app=%a,client=%h '

log_min_duration_statement = 0      # 0 = log EVERY statement with its duration
                                     # use 0 only for an analysis window; then
                                     # return to 1000 for normal operation

log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 0
log_error_verbosity = default
lc_messages = 'C'                  # pgBadger parses English messages
```

### LOG\_MIN\_DURATION\_STATEMENT = 0 IS AN ANALYSIS SETTING, NOT A PRODUCTION SETTING

It logs every statement. On a busy system that is gigabytes per hour, measurable overhead, and personal data written in plain text. Turn it on for a representative window — a peak hour, a batch run — then return it to `1000`. That is the intended workflow.

### Installation

```
dnf install -y perl perl-devel
git clone https://github.com/darold/pgbadger.git && cd pgbadger
perl Makefile.PL && make && sudo make install
pgbadger --version

# or simply
dnf install -y pgbadger      # available in PGDG and EPEL
```

## First reports

```
# One log file
pgbadger /var/lib/pgsql/17/data/log/postgresql-2026-09-14.log -o report.html

# Everything, in parallel, with a prefix override
pgbadger -j 8 -p '%t [%p]: user=%u,db=%d,app=%a,client=%h ' /var/lib/pgsql/17/data/log/postgresql-*.log -o
/var/www/html/pgbadger/report.html

# A specific window
pgbadger -b "2026-09-14 09:00:00" -e "2026-09-14 11:00:00" /var/lib/pgsql/17/data/log/postgresql-*.log -o
peak_hour.html

# Only one database or one user
pgbadger -d xxxx -U xxxx_app postgresql-*.log -o app_only.html

# CSV log input
pgbadger -f csv postgresql.csv -o report.html
```

## Reading the report

| SECTION                    | WHAT TO LOOK AT FIRST                                              |
|----------------------------|--------------------------------------------------------------------|
| Overview                   | Total duration, queries per second, peak times                     |
| Slowest individual queries | The single worst executions — often a report, not the real problem |
| Time-consuming queries     | <b>Start here.</b> Ranked by cumulative time — the real cost       |
| Most frequent queries      | A 3 ms query called a million times                                |
| Temporary files            | Direct evidence that <code>work_mem</code> is too small, per query |
| Locks                      | Which statements waited, and behind what                           |
| Checkpoints                | Frequency, write time, whether forced                              |
| Sessions and connections   | Connection churn — the argument for a pooler                       |
| Events                     | Errors, fatals, deadlocks, cancelled statements                    |

### PGBADGER AND PG\_STAT\_STATEMENTS ARE COMPLEMENTARY

`pg_stat_statements` gives cumulative totals since the last reset, live, with no logging cost. pgBadger gives **time-of-day distribution, actual parameter values, lock waits and errors** — things the view cannot show. Use the view for continuous monitoring and pgBadger for incident analysis.

## 34 pgBadger 2 — Advanced query performance reporting

### CORE

#### Incremental mode — the way to run it in production

```
# Process only what is new since the last run, and build a browsable archive
pgbadger -I -O /var/www/html/pgbadger/ -j 8 /var/lib/pgsql/17/data/log/postgresql-*.log

# Creates:
# /var/www/html/pgbadger/2026/09/14/      per-day reports
# /var/www/html/pgbadger/index.html        a calendar view
# .pgbadger_last_state_file                the incremental marker
```

```
# A nightly cron job
0 1 * * * /usr/bin/pgbadger -I -q -j 4 -O /var/www/html/pgbadger/ -R 30
/var/lib/pgsql/17/data/log/postgresql-*.log
# -R 30 : retain 30 days of reports
```

#### Options that change what you see

```
--top 50                # how many queries per ranking (default 20)
--average 1              # per-minute granularity on the charts
--exclude-query "^SELECT 1$" # drop health checks and noise
--exclude-query "^COMMIT$"
--include-query "xxxx_orders" # focus on one table
--exclude-time "2026-09-14 02:.*" # ignore the backup window
--disable-error --disable-lock # trim the report
--dbname xxxx --dbuser xxxx_app
--histo-granularity 5     # minutes per histogram bucket
--normalized-only         # query shapes only, no parameter values
--anonymize               # REDACT literal values – use when sharing externally
--format json -o report.json # machine-readable output
--quiet --outfile report.html
```

#### Multiple servers in one report

```
# Combine logs from a primary and its standbys
pgbadger -j 8 /logs/node-a/postgresql-*.log /logs/node-b/postgresql-*.log -o cluster_report.html

# Or keep them separate but under one index
pgbadger -I -O /var/www/html/pgbadger/ --prefix-db node-a /logs/node-a/*.log
```

#### Reading the output like an engineer

| OBSERVATION IN THE REPORT                                                              | WHAT IT ACTUALLY MEANS                                                                                   |
|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| One query dominates <i>Time-consuming queries</i> but is not in <i>Slowest queries</i> | High frequency, low individual cost. Fix by <b>calling it less</b> or caching — not by tuning the query. |
| Large <i>Temporary files</i> attributed to one query                                   | <code>work_mem</code> is too small <b>for that query</b> . Raise it per session, not globally.           |
| Lock waits clustered at one time of day                                                | A batch job colliding with online traffic. Reschedule before tuning.                                     |
| Checkpoint write time approaching the interval                                         | <code>max_wal_size</code> too small, or <code>checkpoint_completion_target</code> too low.               |



| OBSERVATION IN THE REPORT                                             | WHAT IT ACTUALLY MEANS                                                       |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------|
| Thousands of connections per minute                                   | No connection pooling. This is a PgBouncer argument, not a database problem. |
| Many <code>ERROR: canceling statement due to statement timeout</code> | Either the timeout is too aggressive, or those queries genuinely need fixing |
| Queries appearing only after a certain hour                           | A scheduled job. Correlate with your own cron and application schedules.     |

### Normalisation, and why the numbers add up differently

pgBadger normalises queries by replacing literals with placeholders, so `WHERE id = 42` and `WHERE id = 43` become one entry. That is what makes the cumulative rankings meaningful. It also means the *Slowest individual queries* section — which shows real executions with real parameters — is where you go to reproduce a problem, while the normalised sections are where you go to prioritise.

```
# Serve the reports
dnf install -y httpd
systemctl enable --now httpd
# then browse http://server/pgbadger/
```

## 35 pgBadger 3 — Production case studies

### CORE

Four patterns that recur, and how the report leads to the fix.

#### Case 1 — “The database is slow every morning at 09:15”

```
pgbadger -b "2026-09-14 09:00:00" -e "2026-09-14 09:45:00" --average 1 postgresql-2026-09-14.log -o morning.html
```

**What the report showed:** a spike in *Lock waits* at exactly 09:15, all waiting on the same table, and a single `ALTER TABLE` in the *DDL* section at 09:14:58.

**Diagnosis:** a deployment script ran DDL during business hours. The `ALTER TABLE` queued for an `ACCESS EXCLUSIVE` lock behind a long `SELECT`, and **every subsequent query queued behind the ALTER** — including plain reads that would otherwise have been compatible.

**Fix:** move DDL to a maintenance window, and set `lock_timeout` in the deployment session so it fails fast instead of freezing the table:

```
SET lock_timeout = '3s';
ALTER TABLE xxxx_orders ADD COLUMN note text;
```

#### Case 2 — “CPU is at 100% but no query looks slow”

**What the report showed:** the top entry under *Most frequent queries* executed 4.2 million times in an hour at a 1.8 ms mean — and appeared nowhere in *Slowest queries*. Under *Time-consuming queries* it accounted for 61% of total database time.

**Diagnosis:** an N+1 query pattern — the application fetched a list, then issued one query per row.

**Fix:** an application change, not a database one. No amount of index or parameter tuning addresses four million unnecessary round trips. This is the case that most justifies pgBadger: `pg_stat_statements` would have shown the same totals, but the time-of-day chart is what convinced the application team.

#### Case 3 — “Disk I/O spikes every few minutes”

```
grep "checkpoint complete" postgresql-*.log | tail -20
# LOG: checkpoint complete: wrote 189244 buffers (72.2%);
#       write=41.2 s, sync=18.7 s, total=60.4 s; ... distance=1048576 kB
```

**What the report showed:** the *Checkpoints* chart showed checkpoints every 3–4 minutes, almost all marked as *requested* rather than *timed*, each writing over 70% of the buffer cache.

**Diagnosis:** `max_wal_size` was still at the 1 GB default under a heavy write workload. WAL filled long before `checkpoint_timeout`, forcing a checkpoint that flushed most of `shared_buffers` at once.

```
ALTER SYSTEM SET max_wal_size = '16GB';
ALTER SYSTEM SET checkpoint_timeout = '20min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
SELECT pg_reload_conf();
```

**Result:** checkpoints moved from every 3 minutes to every 20, `forced_pct` fell from 94% to under 5%, and the I/O spikes disappeared. Total WAL written also fell, because fewer checkpoints means fewer full-page writes.

#### Case 4 — “A report that used to take 20 seconds now takes 9 minutes”

**What the report showed:** the query appeared under *Queries generating the most temporary files*, writing 2.4 GB per execution. Six months earlier it wrote none.

**Diagnosis:** the data had grown past the point where the sort fitted in `work_mem`. The plan had not changed; the sort had silently moved from memory to disk.

```
-- Confirm in the plan
EXPLAIN (ANALYZE, BUFFERS) SELECT ... ;
--   Sort Method: external merge  Disk: 2458112kB          <- the smoking gun

-- Fix: raise work_mem for THAT session only
ALTER ROLE xxxx_report SET work_mem = '512MB';
```

### A repeatable analysis routine

1. Set `log_min_duration_statement = 0` for one representative window — peak hour, or the batch run.
2. Restore it to `1000` immediately afterwards.
3. Run pgBadger over exactly that window with `--average 1`.
4. Open **Time-consuming queries** first, not *Slowest*.
5. Check **Temporary files** and **Locks** before changing any parameter.
6. Change one thing. Re-run the same window next day. Compare.

#### THE DISCIPLINE THAT MAKES THIS WORK

Always analyse the same window, at the same granularity, before and after. A report taken during a quiet afternoon and compared against one from a Monday morning tells you about the traffic, not about your change.

## PHASE 4

# Cloud and DevOps

Sessions 36–40

---

AWS RDS and Aurora, then infrastructure as code, configuration management, schema lifecycle and CI/CD.

**36** AWS-1: Infrastructure Setup — VPC, Security Groups for RDS

---

**37** AWS-2: Migrations to RDS/Aurora using DMS and ora2pg

---

**38** DevOps 1: Terraform — Infrastructure as code

---

**39** DevOps 2: Ansible, Liquibase, Git — Configuration and schema lifecycle

---

**40** DevOps 3: Jenkins — CI/CD orchestration

---

## 36 AWS-1: Infrastructure Setup — VPC, Security Groups for RDS

### TRACK 2 ONLY

**The database is the last thing you create.** Everything before it is network, and getting the network wrong is the most common reason an RDS instance is unreachable, insecure, or impossible to fail over.

| VPC 10.20.0.0/16                                              |  |                           |  |
|---------------------------------------------------------------|--|---------------------------|--|
| AZ ap-south-2a                                                |  | AZ ap-south-2b            |  |
| +-----+                                                       |  | +-----+                   |  |
| PUBLIC 10.20.1.0/24                                           |  | PUBLIC 10.20.2.0/24       |  |
| NAT GW, bastion, ALB                                          |  | NAT GW                    |  |
| +-----+                                                       |  | +-----+                   |  |
| PRIVATE-APP 10.20.11.0/24                                     |  | PRIVATE-APP 10.20.12.0/24 |  |
| application servers                                           |  | application servers       |  |
| +-----+                                                       |  | +-----+                   |  |
| PRIVATE-DB 10.20.21.0/24                                      |  | PRIVATE-DB 10.20.22.0/24  |  |
| RDS writer                                                    |  | RDS standby / reader      |  |
| NO route to 0.0.0.0/0                                         |  | NO route to 0.0.0.0/0     |  |
| +-----+                                                       |  | +-----+                   |  |
| DB SUBNET GROUP spans both DB subnets – REQUIRED for Multi-AZ |  |                           |  |

### Build order

```
aws ec2 create-vpc --cidr-block 10.20.0.0/16 --region ap-south-2 --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=xxxx-vpc}]'

aws ec2 create-subnet --vpc-id vpc-xxx --cidr-block 10.20.21.0/24 --availability-zone ap-south-2a --region ap-south-2
aws ec2 create-subnet --vpc-id vpc-xxx --cidr-block 10.20.22.0/24 --availability-zone ap-south-2b --region ap-south-2

# A DB subnet group must contain subnets in at least TWO AZs
aws rds create-db-subnet-group --db-subnet-group-name xxxx-db-subnets --db-subnet-group-description "xxxx private DB subnets" --subnet-ids subnet-aaa subnet-bbb --region ap-south-2
```

### Security groups — reference groups, never CIDRs

```
aws ec2 create-security-group --group-name xxxx-db-sg --description "PostgreSQL access" --vpc-id vpc-xxx --region ap-south-2

# Allow ONLY the application security group, on 5432
aws ec2 authorize-security-group-ingress --group-id sg-db --protocol tcp --port 5432 --source-group sg-app --region ap-south-2

# And the bastion, for DBA access
aws ec2 authorize-security-group-ingress --group-id sg-db --protocol tcp --port 5432 --source-group sg-bastion --region ap-south-2
```

#### TWO RULES THAT ARE NOT NEGOTIABLE

1. `PubliclyAccessible = false`. A publicly accessible RDS instance is scanned within minutes of creation.
2. Reference **security groups** as the source, never CIDR ranges. Instances change IPs; group membership does not, and the rule documents intent.

## Creating the instance

```
# RDS PostgreSQL, Multi-AZ
aws rds create-db-instance --db-instance-identifier xxxx-prod --db-instance-class db.m6g.xlarge --engine
postgres --engine-version 17.2 --master-username xxxadmin --manage-master-user-password --allocated-storage
500 --max-allocated-storage 2000 --storage-type gp3 --storage-encrypted --db-subnet-group-name xxxx-db-
subnets --vpc-security-group-ids sg-db --db-parameter-group-name xxxx-pg17 --backup-retention-period 14
--preferred-backup-window "18:00-19:00" --preferred-maintenance-window "sun:19:30-sun:20:30" --multi-az --
no-publicly-accessible --enable-performance-insights --performance-insights-retention-period 7 --monitoring-
interval 60 --monitoring-role-arn arn:aws:iam:...:role/rds-monitoring --deletion-protection --region ap-
south-2
```

```
# Aurora PostgreSQL – cluster first, then instances
aws rds create-db-cluster --db-cluster-identifier xxxx-aurora --engine aurora-postgresql --engine-version
16.4 --master-username xxxadmin --manage-master-user-password --db-subnet-group-name xxxx-db-subnets --vpc-
security-group-ids sg-db --backup-retention-period 14 --storage-encrypted --db-cluster-parameter-group-name
xxxx-aurora-cluster-pg --deletion-protection --region ap-south-2

aws rds create-db-instance --db-instance-identifier xxxx-aurora-1 --db-cluster-identifier xxxx-aurora --engine
aurora-postgresql --db-instance-class db.r6g.large --region ap-south-2
```

## RDS versus Aurora, stated honestly

|                 | RDS POSTGRESQL                       | AURORA POSTGRESQL                                    |
|-----------------|--------------------------------------|------------------------------------------------------|
| Storage         | EBS, provisioned size                | Distributed, 6 copies across 3 AZs, auto-grows       |
| Replication     | Standard streaming                   | Storage-layer; replicas share one volume             |
| Failover        | 60–120 s                             | Typically under 30 s                                 |
| Read replicas   | Up to 5, own storage                 | Up to 15, shared storage, near-zero lag              |
| Cost            | Lower at small scale                 | Higher baseline; I/O-priced or I/O-optimized         |
| Engine fidelity | Standard PostgreSQL                  | Modified — some extensions and settings differ       |
| Suits           | Predictable workloads, tight budgets | High read concurrency, fast failover, elastic growth |

## Parameter groups

```
# You cannot edit a default.* group. Create your own before you need to.
aws rds create-db-parameter-group --db-parameter-group-name xxxx-pg17 --db-parameter-group-family postgres17 -
-description "xxxx production" --region ap-south-2

aws rds modify-db-parameter-group --db-parameter-group-name xxxx-pg17 --region ap-south-2 --parameters
"ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate"
"ParameterName=max_locks_per_transaction,ParameterValue=512,ApplyMethod=pending-reboot"
"ParameterName=shared_preload_libraries,ParameterValue=pg_stat_statements,ApplyMethod=pending-reboot"
```

**ApplyType** `static` means a reboot is required; `dynamic` takes effect immediately. Aurora splits parameters between a **cluster** parameter group and an **instance** parameter group — check both when a setting is not where you expect it.

## 37 AWS-2: Migrations to RDS/Aurora using DMS and ora2pg

### TRACK 2 ONLY

```

ORACLE 19c (on-premises)
|
| [1] ASSESS      ora2pg --estimate_cost / SCT assessment report
| [2] CONVERT    schema DDL, types, PL/SQL
| [3] LOAD       DMS full load (or CSV + COPY)
| [4] SYNC       DMS CDC keeps the target current
| [5] VALIDATE   row counts, checksums, DMS data validation
| [6] CUTOVER    freeze writes, drain CDC, repoint, reset sequences
v
AWS AURORA POSTGRESQL

```

### Step 1 — Assessment with ora2pg

```

# ora2pg.conf
ORACLE_DSN      dbi:Oracle:host=oracle.host;sid=ORCL;port=1521
ORACLE_USER     system
ORACLE_PWD      *****
SCHEMA          XXXX_APP
TYPE            TABLE

ora2pg -c ora2pg.conf -t SHOW_REPORT --estimate_cost --dump_as_html > assessment.html

```

The report classifies every object by conversion difficulty and gives a person-day estimate. It is the single most useful artefact for planning the migration — and the number it produces for PL/SQL is usually the largest line in the plan.

### Step 2 — Schema conversion

```

ora2pg -c ora2pg.conf -t TABLE      -o tables.sql
ora2pg -c ora2pg.conf -t SEQUENCE    -o sequences.sql
ora2pg -c ora2pg.conf -t INDEX       -o indexes.sql
ora2pg -c ora2pg.conf -t FKEY        -o fkeys.sql
ora2pg -c ora2pg.conf -t VIEW        -o views.sql
ora2pg -c ora2pg.conf -t FUNCTION    -o functions.sql
ora2pg -c ora2pg.conf -t PACKAGE     -o packages.sql # becomes schemas + functions

```

| ORACLE             | POSTGRESQL                 | WATCH FOR                                                                           |
|--------------------|----------------------------|-------------------------------------------------------------------------------------|
| NUMBER(p,s)        | numeric / bigint / integer | <b>Never blanket-map to numeric</b> — it is slow. Map by actual range.              |
| NUMBER(1)          | boolean                    | Where it is really a flag                                                           |
| VARCHAR2(n)        | varchar(n)                 | Oracle counts <b>bytes</b> , PostgreSQL counts <b>characters</b>                    |
| DATE               | timestamp(0)               | Oracle DATE carries a time; PostgreSQL date does not                                |
| CLOB / BLOB        | text / bytea               | Verify content hashes after load                                                    |
| ROWID              | —                          | No equivalent. Replace with a surrogate key.                                        |
| ' ' (empty string) | —                          | <b>Oracle treats it as NULL; PostgreSQL does not.</b> The number one silent defect. |

### Step 3 and 4 — DMS full load plus CDC

```

# Oracle prerequisites for CDC
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA (ALL) COLUMNS;
ALTER DATABASE FORCE LOGGING;

```

```
aws dms create-replication-instance --replication-instance-identifier xxxx-dms --replication-instance-class
dms.r6i.xlarge --allocated-storage 200 --multi-az --replication-subnet-group-identifier xxxx-dms-subnets --
-vpc-security-group-ids sg-dms --region ap-south-2

aws dms create-replication-task --replication-task-identifier xxxx-full-cdc --source-endpoint-arn
arn:...:oracle-src --target-endpoint-arn arn:...:aurora-tgt --replication-instance-arn arn:...:xxxx-dms --
migration-type full-load-and-cdc --table-mappings file://mappings.json --replication-task-settings
file://settings.json
```

```
// settings.json – the parts that matter
{
  "TargetMetadata": { "ParallelLoadThreads": 8, "BatchApplyEnabled": true },
  "FullLoadSettings": {
    "TargetTablePrepMode": "DO_NOTHING",      // schema already created by ora2pg
    "MaxFullLoadSubTasks": 8,
    "CommitRate": 50000
  },
  "ValidationSettings": { "EnableValidation": true, "ThreadCount": 5 },
  "Logging": { "EnableLogging": true }
}
```

### TARGETTABLEPREPMode

Let **ora2pg** create the schema and set DMS to **DO\_NOTHING**. If DMS creates the tables it chooses its own type mappings, which are conservative and frequently wrong for numeric precision. You want your reviewed DDL, not DMS's guess.

## Step 5 — Validation

```
-- Row counts, source versus target, per table
SELECT 'xxxx_orders' AS t, count(*) FROM xxxx_orders;      -- run on both

-- Column aggregates: sums, min, max, nulls, distinct
SELECT sum(amount), min(created_on), max(created_on),
       count(*) FILTER (WHERE status IS NULL) FROM xxxx_orders;

-- The empty-string trap – run this on the TARGET
SELECT count(*) FROM xxxx_orders WHERE some_text = '';     -- should be 0 if source had NULLs

-- Sequences – the classic post-migration production failure
SELECT 'SELECT setval('' || sequencename || '', ' ||
       '(SELECT COALESCE(max(id),1) FROM ' || tablename || ');'
FROM   pg_sequences;  -- generate, review, run

-- DMS's own validation
aws dms describe-table-statistics --replication-task-arn arn:... --query "TableStatistics[?
ValidationState!='Validated']"
```

## Step 6 — Cutover

```
# 1. Freeze application writes on Oracle
# 2. Wait for CDC latency to reach zero
aws dms describe-replication-tasks --query "ReplicationTasks[0].ReplicationTaskStats.
[FullLoadProgressPercent,ElapsedTimeMillis]"
# and CloudWatch: CDCLatencySource, CDCLatencyTarget -> 0
# 3. Final validation queries
# 4. RESET ALL SEQUENCES
# 5. ANALYZE the whole database
vacuumdb --analyze-in-stages --jobs=8 -h aurora.host -d xxxx
# 6. Repoint connection strings
# 7. Keep Oracle read-only as a fallback for an agreed window
```



## 38 DevOps 1: Terraform — Infrastructure as code

### TRACK 2 ONLY

**Why it matters to a DBA.** A database created by clicking is a database nobody can recreate, audit or review. Terraform makes the instance, its parameter group, its subnets and its security rules reviewable artefacts in Git — and makes “rebuild the DR environment” a command rather than a project.

### Project layout

```
xxxx-infra/
  backend.tf           remote state, with locking
  providers.tf
  variables.tf
  network.tf           VPC, subnets, DB subnet group
  security.tf          security groups
  rds.tf               instance / cluster, parameter groups
  monitoring.tf        CloudWatch alarms
  outputs.tf
  envs/
    dev.tfvars
    uat.tfvars
    prod.tfvars
```

### Remote state — do this before anything else

```
# backend.tf
terraform {
  required_version = ">= 1.6"
  backend "s3" {
    bucket      = "xxxx-terraform-state"
    key         = "rds/terraform.tfstate"
    region      = "ap-south-2"
    dynamodb_table = "xxxx-terraform-locks" # prevents concurrent applies
    encrypt     = true
  }
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
```

### STATE IS AS SENSITIVE AS A PASSWORD FILE

The state file contains connection endpoints and, in some resources, secrets in plain text. It must live in an encrypted bucket with restricted access and versioning enabled — never in Git, never on a laptop.

## An RDS instance as code

```

# rds.tf
resource "aws_db_parameter_group" "xxxx" {
  name     = "xxxx-pg17-${var.env}"
  family   = "postgres17"

  parameter { name = "log_min_duration_statement" value = "1000" }
  parameter { name = "log_checkpoints"             value = "1" }
  parameter { name = "log_lock_waits"              value = "1" }
  parameter { name = "shared_preload_libraries"     value = "pg_stat_statements"
    apply_method = "pending-reboot" }
  parameter { name = "max_locks_per_transaction"   value = "512"
    apply_method = "pending-reboot" }
  parameter { name = "random_page_cost"            value = "1.1" }

  lifecycle { create_before_destroy = true }
}

resource "aws_db_instance" "xxxx" {
  identifier      = "xxxx-${var.env}"
  engine          = "postgres"
  engine_version  = var.engine_version
  instance_class  = var.instance_class

  allocated_storage      = var.allocated_storage
  max_allocated_storage  = var.max_allocated_storage
  storage_type           = "gp3"
  storage_encrypted      = true
  kms_key_id             = aws_kms_key.xxxx.arn

  db_name      = "xxxx"
  username     = "xxxxadmin"
  manage_master_user_password = true          # AWS-managed secret; no password in state

  db_subnet_group_name   = aws_db_subnet_group.xxxx.name
  vpc_security_group_ids = [aws_security_group.xxxx_db.id]
  parameter_group_name   = aws_db_parameter_group.xxxx.name
  publicly_accessible    = false

  multi_az          = var.env == "prod"
  backup_retention_period = var.env == "prod" ? 30 : 7
  backup_window      = "18:00-19:00"
  maintenance_window = "sun:19:30-sun:20:30"
  auto_minor_version_upgrade = false          # control your own upgrade timing

  performance_insights_enabled = true
  monitoring_interval           = 60
  monitoring_role_arn           = aws_iam_role.rds_monitoring.arn
  enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

  deletion_protection      = var.env == "prod"
  skip_final_snapshot      = var.env != "prod"
  final_snapshot_identifier = "xxxx-${var.env}-final-${formatdate("YYYYMMDDhhmm", timestamp())}"

  lifecycle { ignore_changes = [final_snapshot_identifier] }
  tags = { Environment = var.env, Application = "xxxx", ManagedBy = "terraform" }
}

```

## Workflow

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -var-file=envs/prod.tfvars -out=prod.tfplan
# REVIEW THE PLAN. Every destroy line on a database resource is a question,
# not a formality.
terraform apply prod.tfplan

terraform state list
terraform show
terraform import aws_db_instance.xxxx xxxx-prod # adopt an existing instance
```

### THE PLAN LINES THAT MUST STOP YOU

`-/+ destroy and then create replacement` on a database resource means Terraform intends to **delete your database and build a new empty one**. Changing `engine_version` across a major, `db_name`, or the availability zone can all trigger it. Read every plan against a production database line by line, and keep `deletion_protection = true` as the backstop.

## 39 DevOps 2: Ansible, Liquibase, Git — Configuration and schema lifecycle

### TRACK 2 ONLY

#### Ansible — configuration management for the servers

```
# inventory.yml
all:
  children:
    postgres_primary:
      hosts: { node-a: { ansible_host: 10.20.30.41 } }
    postgres_standby:
      hosts:
        node-b: { ansible_host: 10.20.30.42 }
        node-c: { ansible_host: 10.20.30.43 }
```

```
# roles/postgres/tasks/main.yml
- name: Install PGDG repository
  ansible.builtin.dnf:
    name: "https://download.postgresql.org/pub/repos/yum/repos/pms/EL-9-x86_64/pgdg-redhat-repo-
latest.noarch.rpm"
    disable_gpg_check: true

- name: Disable the distribution module
  ansible.builtin.command: dnf -qy module disable postgresql
  changed_when: false

- name: Install PostgreSQL
  ansible.builtin.dnf:
    name: [postgresql17-server, postgresql17-contrib, postgresql17-devel]
    state: present

- name: Apply kernel tuning
  ansible.posix.sysctl:
    name: "{{ item.name }}"
    value: "{{ item.value }}"
    sysctl_file: /etc/sysctl.d/30-postgresql.conf
    reload: true
  loop:
    - { name: vm.swappiness, value: '1' }
    - { name: vm.overcommit_memory, value: '2' }
    - { name: vm.dirty_background_ratio, value: '5' }
    - { name: vm.dirty_ratio, value: '10' }

- name: Deploy postgresql.conf from template
  ansible.builtin.template:
    src: postgresql.conf.j2
    dest: "{{ pgdata }}/postgresql.conf"
    owner: postgres
    group: postgres
    mode: '0600'
  notify: reload postgresql

- name: Deploy pg_hba.conf
  ansible.builtin.template:
    src: pg_hba.conf.j2
    dest: "{{ pgdata }}/pg_hba.conf"
    owner: postgres
    mode: '0600'
  notify: reload postgresql
```

```
# templates/postgresql.conf.j2 - values computed from the host's facts
shared_buffers = {{ (ansible_memtotal_mb * 0.25) | int }}MB
effective_cache_size = {{ (ansible_memtotal_mb * 0.70) | int }}MB
maintenance_work_mem = {{ [ (ansible_memtotal_mb * 0.05) | int, 2048 ] | min }}MB
max_connections = {{ pg_max_connections | default(200) }}
max_parallel_workers = {{ ansible_processor_vcpus }}
random_page_cost = {{ pg_random_page_cost | default(1.1) }}
```

```
ansible-playbook -i inventory.yml site.yml --check --diff      # dry run first
ansible-playbook -i inventory.yml site.yml --limit postgres_standby
ansible-vault encrypt group_vars/all/vault.yml                 # secrets never in plain text
```

## Liquibase — schema change as versioned, reviewable artefacts

```
<!-- changelog/001-orders-table.xml -->
<changeSet id="001-create-orders" author="majid">
  <preConditions onFail="MARK_RAN">
    <not><tableExists tableName="xxxx_orders" schemaName="app"/></not>
  </preConditions>
  <createTable tableName="xxxx_orders" schemaName="app">
    <column name="order_id" type="bigint" autoIncrement="true">
      <constraints primaryKey="true" nullable="false"/>
    </column>
    <column name="status" type="varchar(30)"><constraints nullable="false"/></column>
    <column name="created_on" type="timestampz" defaultValueComputed="now()"/>
  </createTable>
  <rollback><dropTable tableName="xxxx_orders" schemaName="app"/></rollback>
</changeSet>

<changeSet id="002-index-created-on" author="majid" runInTransaction="false">
  <sql>CREATE INDEX CONCURRENTLY idx_xxxx_orders_created
    ON app.xxxx_orders (created_on);</sql>
  <rollback>DROP INDEX CONCURRENTLY app.idx_xxxx_orders_created;</rollback>
</changeSet>
```

```
liquibase --changelog-file=changelog/master.xml status
liquibase --changelog-file=changelog/master.xml updateSQL      # REVIEW before applying
liquibase --changelog-file=changelog/master.xml update
liquibase --changelog-file=changelog/master.xml rollbackCount 1
liquibase --changelog-file=changelog/master.xml tag release-2026-09
liquibase --changelog-file=changelog/master.xml rollback release-2026-09
```

### CREATE INDEX CONCURRENTLY NEEDS RUNINTRANSACTION="FALSE"

It cannot run inside a transaction block. Liquibase wraps changesets in transactions by default, so without that attribute the changeset fails. The same applies to `REINDEX CONCURRENTLY`, `VACUUM` and `ALTER TYPE ... ADD VALUE`.

## Git — what a DBA keeps under version control

| REPOSITORY       | CONTENTS                                                          |
|------------------|-------------------------------------------------------------------|
| xxxx-infra       | Terraform: instances, parameter groups, networking, alarms        |
| xxxx-config      | Ansible: roles, templates, inventory, vaulted secrets             |
| xxxx-schema      | Liquibase changelogs — every DDL change, reviewed by pull request |
| xxxx-dba-scripts | Monitoring queries, maintenance jobs, runbooks                    |

```
git checkout -b feature/orders-index
# edit changelog
liquibase updateSQL > review.sql          # attach the generated SQL to the pull request
git commit -am "Add index on xxxx_orders.created_on (concurrently)"
git push -u origin feature/orders-index
# pull request -> DBA review -> merge -> pipeline applies to dev, then uat, then prod
```

#### THE CULTURAL CHANGE THIS REPRESENTS

Schema change stops being a DBA running a script at midnight and becomes a reviewed, tested, reversible artefact that moves through environments on the same path as application code. The DBA's role shifts from executing changes to **reviewing them** — which is where the expertise is actually worth most.

## 40 DevOps 3: Jenkins — CI/CD orchestration

### TRACK 2 ONLY

**What a database pipeline must do differently from an application pipeline.** Application deployments are replaceable; database changes are not. The pipeline therefore needs a backup gate, a review gate, and a rollback path before it is allowed anywhere near production.

```
Git push / pull request merged
|
[1] VALIDATE    liquibase validate, SQL lint, forbidden-pattern scan
|
[2] DEV        apply to dev, run smoke tests
|
[3] REVIEW     post updateSQL output to the PR; DBA approval required
|
[4] UAT        apply to uat, run regression + performance comparison
|
[5] BACKUP     snapshot production BEFORE touching it          <-- gate
|
[6] APPROVE    manual input step, named approver               <-- gate
|
[7] PROD       apply with lock_timeout, capture the tag
|
[8] VERIFY     post-deploy checks; auto-rollback on failure
```

## A declarative pipeline

```

pipeline {
  agent any
  parameters {
    choice(name: 'ENVIRONMENT', choices: ['dev','uat','prod'])
    booleanParam(name: 'DRY_RUN', defaultValue: true)
  }
  environment {
    LIQUIBASE_HOME = '/opt/liquibase'
    PGPASSWORD      = credentials('xxxx-db-password')
  }

  stages {
    stage('Validate') {
      steps {
        sh 'liquibase --changelog-file=changelog/master.xml validate'
        sh '''
          # Refuse destructive DDL that has not been explicitly approved
          if grep -riE "DROP (TABLE|COLUMN|SCHEMA)|TRUNCATE" changelog/ \
            | grep -v "APPROVED-DESTRUCTIVE"; then
            echo "Destructive DDL without approval marker"; exit 1
          fi
          # Refuse a non-concurrent index build on a large table
          if grep -riE "CREATE INDEX (?!CONCURRENTLY)" changelog/; then
            echo "Use CREATE INDEX CONCURRENTLY"; exit 1
          fi
          '''
      }
    }

    stage('Generate SQL for review') {
      steps {
        sh 'liquibase --changelog-file=changelog/master.xml updateSQL > review.sql'
        archiveArtifacts artifacts: 'review.sql'
      }
    }

    stage('Apply to dev') {
      when { expression { params.ENVIRONMENT in ['dev','uat','prod'] } }
      steps { sh 'liquibase --defaults-file=liquibase.dev.properties update' }
    }

    stage('Backup production') {
      when { expression { params.ENVIRONMENT == 'prod' } }
      steps {
        sh '''
          aws rds create-db-snapshot \
            --db-instance-identifier xxxx-prod \
            --db-snapshot-identifier xxxx-prod-predeploy-${BUILD_NUMBER} \
            --region ap-south-2
          aws rds wait db-snapshot-available \
            --db-snapshot-identifier xxxx-prod-predeploy-${BUILD_NUMBER} \
            --region ap-south-2
          '''
      }
    }

    stage('Approval') {
      when { expression { params.ENVIRONMENT == 'prod' } }
      steps {
        timeout(time: 60, unit: 'MINUTES') {
          input message: 'Apply to PRODUCTION?', submitter: 'dba-team',
            parameters: [text(name: 'CHANGE_TICKET', defaultValue: '')]
        }
      }
    }

    stage('Apply to production') {
      when { expression { params.ENVIRONMENT == 'prod' } }
    }
  }
}

```



```

    steps {
      sh '''
        liquibase --defaults-file=liquibase.prod.properties \
          tag predeploy-${BUILD_NUMBER}
        PGOPTIONS="-c lock_timeout=5s -c statement_timeout=300s" \
          liquibase --defaults-file=liquibase.prod.properties update
      '''
    }
  }

  stage('Verify') {
    steps {
      sh '''
        psql -h $DB_HOST -d xxxx -v ON_ERROR_STOP=1 -f tests/post_deploy_checks.sql
      '''
    }
  }
}

post {
  failure {
    sh '''
      if [ "${ENVIRONMENT}" = "prod" ]; then
        liquibase --defaults-file=liquibase.prod.properties \
          rollback predeploy-${BUILD_NUMBER}
      fi
    '''
    mail to: 'dba-team@example.com',
        subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        body: "Rollback attempted. See ${env.BUILD_URL}"
  }
  success { echo "Deployed to ${params.ENVIRONMENT}" }
}
}

```

## PGOPTIONS — the safety net that belongs in every pipeline

```
PGOPTIONS="-c lock_timeout=5s -c statement_timeout=300s -c idle_in_transaction_session_timeout=60s"
```

Without `lock_timeout`, a migration's `ALTER TABLE` queues behind a long-running query — and every query behind it queues too, freezing the table. With it, the migration fails in five seconds and the pipeline retries later. **That one environment variable is the difference between a failed deployment and an outage.**

## What the post-deploy checks should assert

```

-- tests/post_deploy_checks.sql
\set ON_ERROR_STOP on
DO $$ BEGIN
  ASSERT (SELECT count(*) FROM information_schema.tables
    WHERE table_schema='app' AND table_name='xxxx_orders') = 1,
    'xxxx_orders missing';
  ASSERT (SELECT count(*) FROM pg_index WHERE NOT indisvalid) = 0,
    'invalid index present';
  ASSERT (SELECT count(*) FROM pg_stat_activity
    WHERE state='idle in transaction'
      AND now()-state_change > interval '5 min') = 0,
    'sessions left idle in transaction';
END $$;

```

## PHASE 5

# Observability and Wrap-Up

Sessions 41–42

---

Knowing before the users tell you, and what to build next.

**41** Observability — Grafana, Prometheus, dashboards and alerting

---

**42** Conclusion — Recap, next steps, what to build next

---

## 41 Observability — Grafana, Prometheus, dashboards and alerting

CORE

```

PostgreSQL --> postgres_exporter --> Prometheus --> Grafana
                :9187                scrape /15s                dashboards
                                |
                                Alertmanager --> Slack / PagerDuty / email

node_exporter (:9100) --> Prometheus      CPU, memory, disk, network
pgbouncer_exporter   --> Prometheus      pool saturation

```

### Setting up the exporter

```

-- A least-privilege monitoring role
CREATE ROLE xxxx_exporter LOGIN PASSWORD '...' CONNECTION LIMIT 5;
GRANT pg_monitor TO xxxx_exporter;          -- PG 10+: exactly the right grant
GRANT CONNECT ON DATABASE xxxx TO xxxx_exporter;

```

```

# /etc/default/postgres_exporter
DATA_SOURCE_NAME="postgresql://xxxx_exporter:PASS@localhost:5432/postgres?sslmode=require"
PG_EXPORTER_EXTEND_QUERY_PATH=/etc/postgres_exporter/queries.yaml
PG_EXPORTER_AUTO_DISCOVER_DATABASES=true

```

```

# queries.yaml – custom metrics the default exporter does not provide
pg_replication_slots:
  query: |
    SELECT slot_name, active::int AS active,
           pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS retained_bytes
    FROM pg_replication_slots
  metrics:
    - slot_name: { usage: LABEL }
    - active: { usage: GAUGE, description: "Slot is active" }
    - retained_bytes: { usage: GAUGE, description: "WAL retained by this slot" }

pg_lock_slots:
  query: |
    SELECT (SELECT count(*) FROM pg_locks) AS locks_held,
           current_setting('max_locks_per_transaction')::int
           * current_setting('max_connections')::int AS lock_slots
  metrics:
    - locks_held: { usage: GAUGE }
    - lock_slots: { usage: GAUGE }

pg_xid_age:
  query: SELECT max(age(datfrozenxid)) AS max_xid_age FROM pg_database
  metrics:
    - max_xid_age: { usage: GAUGE, description: "Transaction ID age" }

```

## The alert rules that matter

```
# /etc/prometheus/rules/postgres.yml
groups:
- name: postgresql
  rules:
  - alert: PostgresDown
    expr: pg_up == 0
    for: 1m
    labels: { severity: critical }
    annotations: { summary: "PostgreSQL is down on {{ $labels.instance }}" }

  - alert: ConnectionsNearLimit
    expr: sum(pg_stat_activity_count) by (instance)
      / on(instance) pg_settings_max_connections > 0.85
    for: 5m
    labels: { severity: critical }

  - alert: ReplicationLagHigh
    expr: pg_replication_lag_seconds > 300
    for: 5m
    labels: { severity: critical }

  - alert: InactiveReplicationSlot
    expr: pg_replication_slots_active == 0
    for: 10m
    labels: { severity: warning }
    annotations: { summary: "Inactive slot {{ $labels.slot_name }} is retaining WAL" }

  - alert: TransactionIDWraparoundRisk
    expr: pg_xid_age_max_xid_age > 1000000000
    for: 30m
    labels: { severity: critical }

  - alert: LockSlotsHigh
    expr: pg_lock_slots_locks_held / pg_lock_slots_lock_slots > 0.6
    for: 5m
    labels: { severity: warning }

  - alert: CacheHitRatioLow
    expr: rate(pg_stat_database_blks_hit[5m])
      / (rate(pg_stat_database_blks_hit[5m])
        + rate(pg_stat_database_blks_read[5m])) < 0.90
    for: 15m
    labels: { severity: warning }

  - alert: DiskSpaceLow
    expr: node_filesystem_avail_bytes{mountpoint="/var/lib/pgsql"}
      / node_filesystem_size_bytes{mountpoint="/var/lib/pgsql"} < 0.15
    for: 5m
    labels: { severity: critical }
```

## Dashboard design

| ROW           | PANELS                                                                   |
|---------------|--------------------------------------------------------------------------|
| Availability  | Up/down, uptime, connections used versus max, connection states          |
| Throughput    | Transactions per second, rows returned/fetched/inserted/updated/deleted  |
| Latency       | Mean and p95 from <code>pg_stat_statements</code> , active session count |
| Cache and I/O | Cache hit ratio, block reads, temp bytes written                         |
| Replication   | Lag in bytes and seconds per standby, slot retention, standby count      |
| Maintenance   | Dead tuples by table, autovacuum runs, XID age, checkpoint frequency     |

| ROW      | PANELS                                                              |
|----------|---------------------------------------------------------------------|
| Capacity | Database size trend, largest tables, disk free, WAL generation rate |
| Locks    | Blocked sessions, lock slot utilisation, deadlock rate              |

Grafana dashboard IDs **9628** (PostgreSQL Database) and **455** are sound starting points; import one, then replace its panels with the metrics your alerts actually fire on. A dashboard whose panels do not correspond to alerts is decoration.

#### ALERT ON SYMPTOMS, GRAPH CAUSES

Page on **connections near limit, replication lag, disk low, wraparound risk, instance down** — things that require action now. Everything else belongs on a dashboard you consult once the page has arrived. An alert that nobody acts on trains people to ignore all of them.

## 42 Conclusion — Recap, next steps, what to build next

### TRACK 2 ONLY

What the forty-one preceding sessions add up to

| PHASE     | CAPABILITY GAINED                          | THE QUESTION YOU CAN NOW ANSWER                           |
|-----------|--------------------------------------------|-----------------------------------------------------------|
| 1 (1–20)  | Architecture, installation, administration | <i>Why does PostgreSQL behave this way?</i>               |
| 2 (21–29) | Backup, replication, high availability     | <i>What happens when it breaks?</i>                       |
| 3 (30–35) | Parameter tuning and log analysis          | <i>Why is it slow, and what specifically do I change?</i> |
| 4 (36–40) | Cloud and DevOps                           | <i>How do I run this repeatably, at scale?</i>            |
| 5 (41)    | Observability                              | <i>How do I know before the users tell me?</i>            |

The habits that separate a DBA from an operator

- **Diagnose in order.** Sessions, then blocking, then long transactions, then queries, then the machine. Never change a parameter before you know which layer is at fault.
- **A backup that has never been restored is not a backup.** Rehearse monthly, record the elapsed time, and call that number your RTO.
- **Fence before you promote.** Every split-brain incident begins with a promotion that skipped this.
- **State the trade-off, then choose.** `hot_standby_feedback` moves cost from the standby to the primary. `--link` trades rollback for speed. `max_slot_wal_keep_size` sacrifices a standby to protect the primary. Naming the trade is the senior answer.
- **Change one thing, measure, keep or revert.** A configuration you cannot explain is a configuration you cannot defend.
- **If VACUUM runs but frees nothing, something holds the xmin horizon.** A long transaction, a replication slot, or a prepared transaction — always one of the three.

What to build next — a portfolio, not a certificate

| # | BUILD THIS                                                                                                                                       | BECAUSE IT PROVES                        |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 1 | A three-node Patroni cluster on VMs, with etcd, HAProxy and PgBouncer — then <b>kill the primary</b> and time the failover                       | You have operated HA, not read about it  |
| 2 | A pgBackRest repository with full, differential and incremental backups, and a <b>scripted monthly restore rehearsal</b> that asserts row counts | You treat backups as a tested system     |
| 3 | A PITR drill: drop a table, recover to one second before, document the elapsed time                                                              | You can recover under pressure           |
| 4 | Terraform for RDS plus Ansible for a self-managed pair, both in Git with CI                                                                      | You can build the same environment twice |

| # | BUILD THIS                                                                                                                                          | BECAUSE IT PROVES                        |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 5 | Prometheus, Grafana and Alertmanager with the eight alert rules from Session 41 actually firing                                                     | You monitor symptoms, not vanity metrics |
| 6 | Take a deliberately badly configured instance, use pgBadger and <code>pg_stat_statements</code> to fix it, and <b>write up the before and after</b> | You can demonstrate measured improvement |
| 7 | An Oracle to PostgreSQL migration of a small real schema, using ora2pg and DMS, with a reconciliation report                                        | You can run a migration end to end       |

## Where to keep learning

| SOURCE                                                                  | WHY                                                                                   |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| The official documentation                                              | Genuinely excellent, and the authoritative answer for every version-specific question |
| Release notes for every major version                                   | The only place incompatibilities are listed before they bite you                      |
| <code>pgsql-general</code> and <code>pgsql-hackers</code> mailing lists | Where the decisions are actually made and explained                                   |
| Planet PostgreSQL                                                       | Aggregated practitioner blogs                                                         |
| PGConf, PGDay, local user groups                                        | The people who wrote the features are approachable in person                          |
| The source tree's <code>README</code> files                             | <code>src/backend/access/nbtree/README</code> and similar are better than most books  |

### THE LAST WORD

Production PostgreSQL is not a body of trivia. It is a small number of mechanisms — MVCC, WAL, the planner, the process model — and a habit of reasoning from them rather than from memorised symptoms. Everything in these forty-two sessions reduces to that. When you meet a problem this course did not cover, the mechanisms will still be there, and they will still explain it.